


欢迎选修《数据结构》

- 课程编号：30240184
- 授课教师：邓俊辉、尹 霞、陈键飞
- 开课单位：计算机系

2026秋季《数据结构》尹霞老师课堂课表微调

(30240184-2，二教402)

2026秋周次	周三	周五	周日
第1周	9月16日(2学时) DSA(08:00-09:35)	9月18日(**3学时**) DSA(09:50-12:15)	9月20日(**3学时**) DSA(09:50-12:15)
第2周	9月23日(2学时) DSA(08:00-09:35)	中秋节 课程调整到9月20日	
第3周	9月30日(2学时) DSA(08:00-09:35)	国庆节放假	
第4周	国庆节放假	10月9日(2学时) DSA(09:50-11:25)	
第5-15周	(2学时) DSA(08:00-09:35)	(2学时) DSA(09:50-11:25)	**第8周(11月8日上午)** **期中考试**
第16周	**已调课完成** (9月18日+9月20日)	元旦放假	

《数据结构与算法》DSA学什么？

线性结构

- 01.Introduction
  用尺子丈量
- 02.Vector
  列表的排序
- 03.List
  向量的排序
- 04.Stack+Quene
  各类应用

半线性结构

- 05.Binary_Tree
  树的遍历
- 06.Binary_Search_Tree
- 07.BST_Application
- 08.Advanced_BST
  各种树及中序遍历
- 09.Dictionary
  小容器放下大空间

非线性结构

- 10.Graph
- 11.Graph_Application
  图的遍历
- 12.PQ
  只关注极值的“树”
- 13.String
  最常见的串匹配
- 14.Sorting
  永恒的排序算法

1．绪论

A.计算

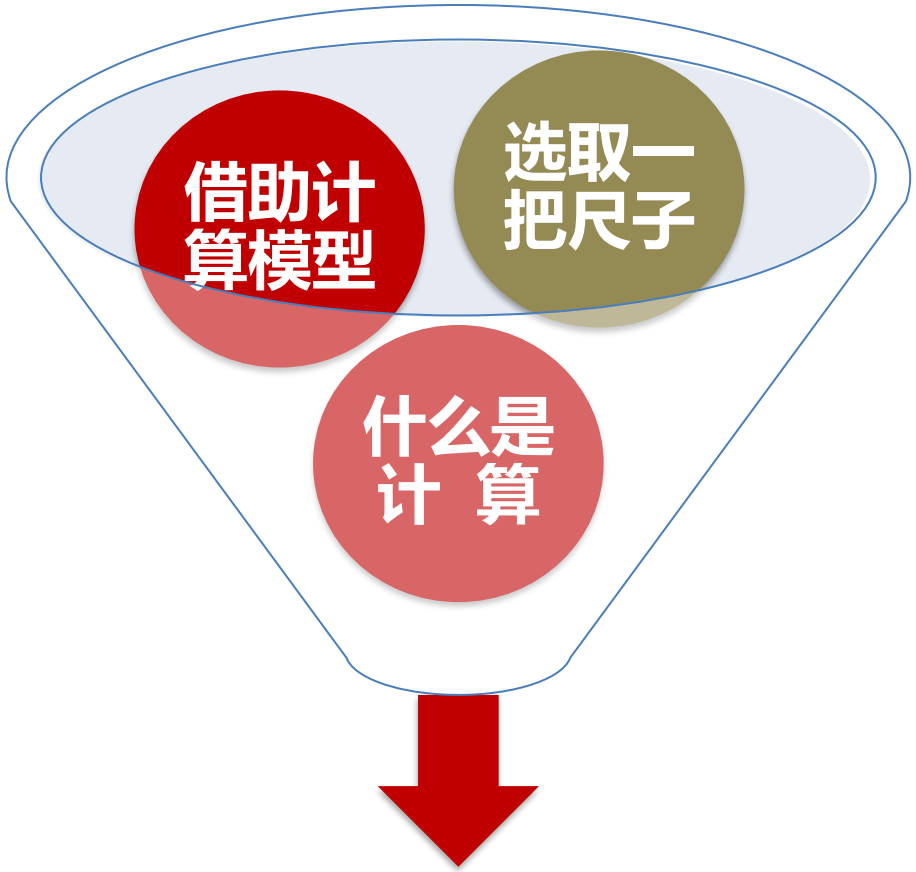
B.计算模型

C.渐进复杂度

D.复杂度分析

E.迭代与递归

F. 动态规划



开 始 量 来 量 去 ……

迭代与递归、级数求和、递归跟踪、递推方程

1. 绪论

01-A.计算

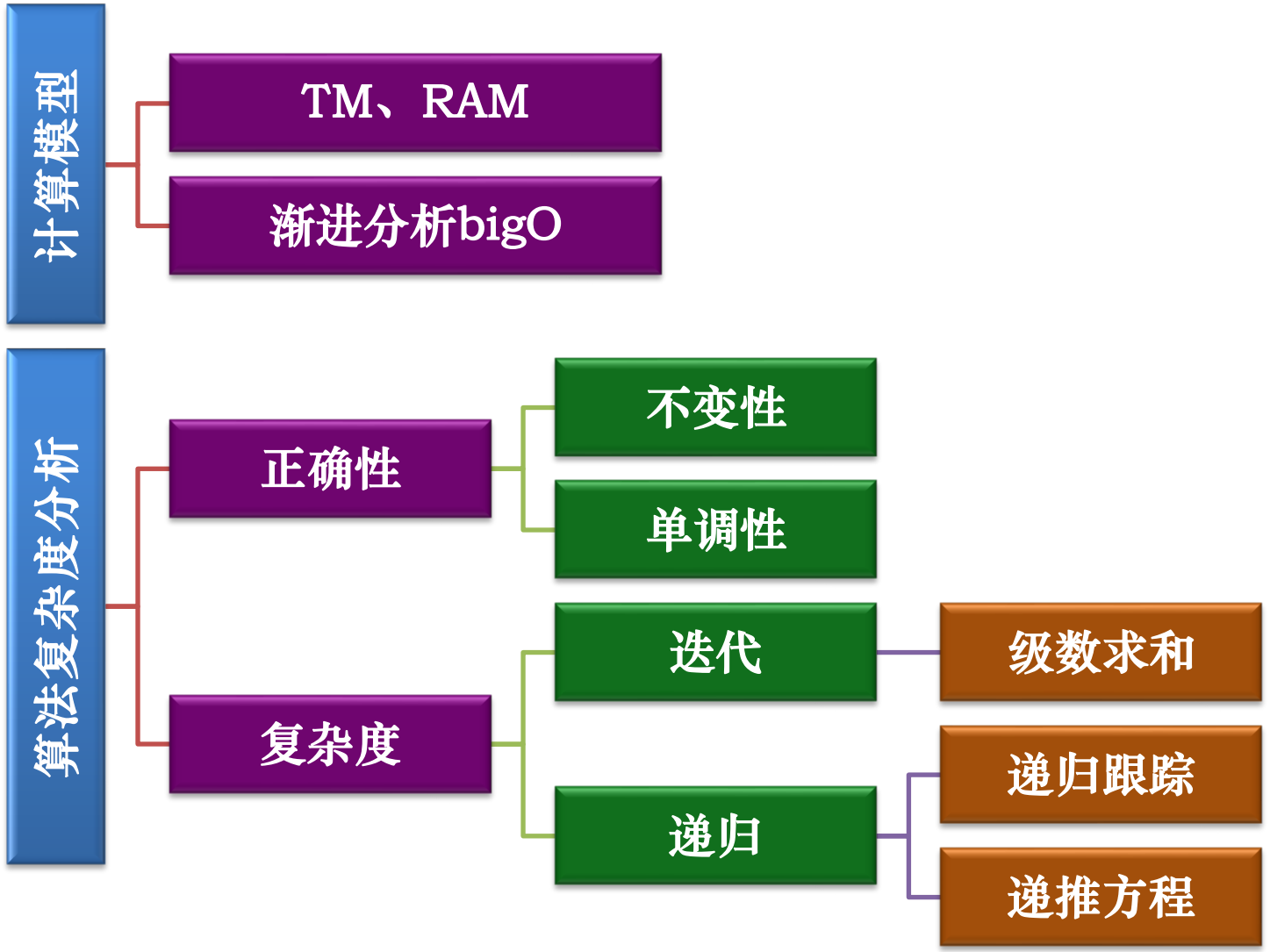
01-B.计算模型

01-C.渐进复杂度

01-D.复杂度分析

01-E.迭代与递归

01-F. 动态规划



01.Introduction

A-计算

绪论

从计算，到算法

邓俊辉
deng@tsinghua.edu.cn

曹冲称象：船、水、石

置象大船之上，而

刻其水痕所至





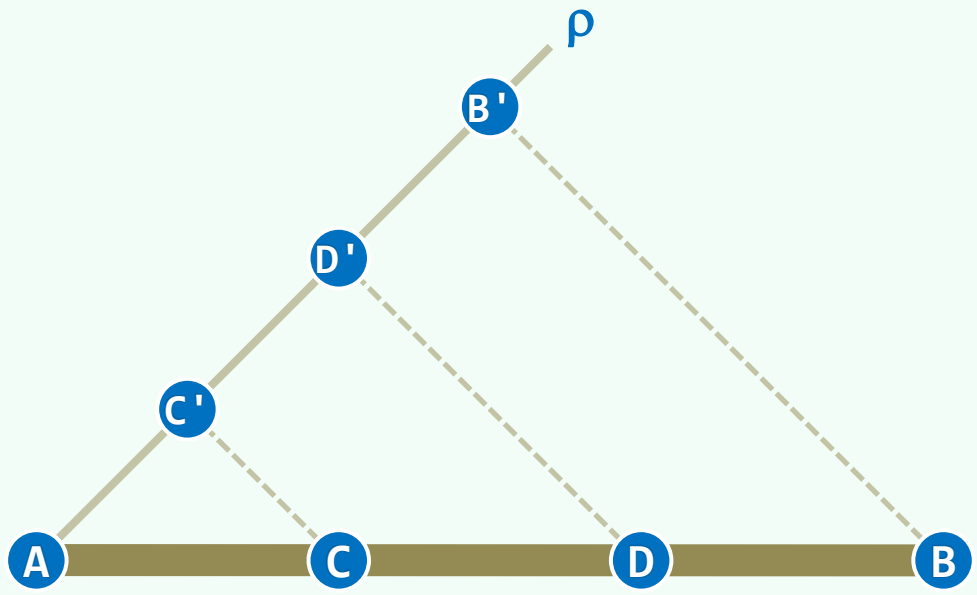
称物以**载**之，则

校可知矣

鄧哀王沖字倉舒。少聰察岐嶷，生五六歲，智意所及，有若成人之智。時孫權曾致巨象，太祖欲知其斤重，訪之羣下，咸莫能出其理。沖曰：「置象大船之上，而刻其水痕所至，稱物以載之，則校可知矣。」太祖大悅，即施行焉。時軍國多事，用刑嚴重。太祖馬鞍在

欧几里得：尺规计算机

- 任给平面上线段AB（输入），将其三等分（输出）
  - 从A发出一条与AB不重合的射线ρ
  - 在ρ上取 |AC'| = |C'D'| = |D'B'|
  - 联接B'B
  - 经D'做B'B的平行线，交AB于D
  - 经C'做B'B的平行线，交AB于C
- 这里的计算机是什么？
- 它能够解决什么问题？不能解决什么问题？
- 子程序：过直线外一点，做平行线



算法的要素

- 计算 = 信息处理 = 借助某种工具，遵照一定规则，以明确而机械的形式进行
- 计算模型 = 计算机 = 信息处理工具
- 所谓算法，即特定计算模型下，旨在解决特定问题的指令序列

input	输入	待处理的信息（问题）
output	输出	经处理的信息（答案）
correctness	正确	的确可以解决指定的问题
definiteness	明确	可描述为一个由基本操作组成的序列 //加盐少许？加糖适量？煮至半熟？
effectiveness	可行	每一基本操作都可实现，且在常数时间内完成 //把大象放进冰箱？
finiteness	有穷	对于任何输入，经有穷次基本操作，都可以得到输出

... ...

有穷性：程序~算法

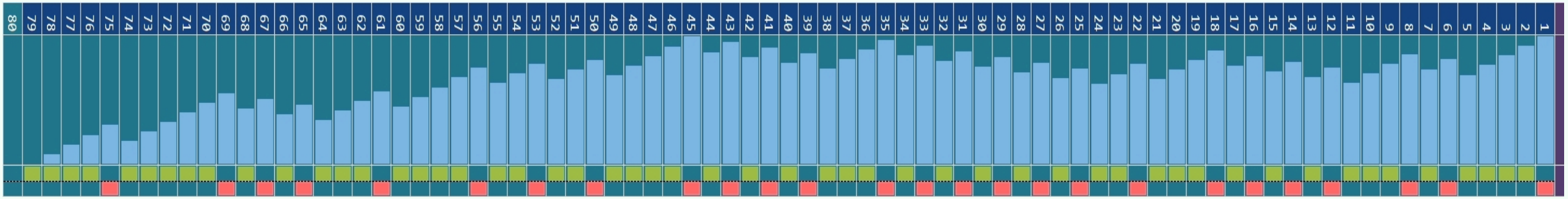
❖ Hailstone序列

$$\mathcal{H}(n) = \begin{cases} \{1\} & (n \le 1) \\ \{n\} \cup \mathcal{H}(n/2) & (n \text{ is even}) \\ \{n\} \cup \mathcal{H}(3n+1) & (n \text{ is odd}) \end{cases}$$

```
int hailstone( int n ) {
   int length = 1;
   for ( ; 1 < n; length++ )
      n = n%2 ? 3*n+1 : n/2;
   return length;
}
```

❖ 对于任意 $n > 0$，总有 $|\mathcal{H}(n)| < \infty$

❖ Paul Erdos:Mathematics is NOT yet ready for such problems



有穷性：理想硬币～几何分布～数学期望

❖ 假设：rand()为理想的随机整数发生器

于是：rand()为奇、偶的概率均为50%

❖ 考查 ~~算法~~ 程序：`void toss() { while ( rand() & 1 ); }`

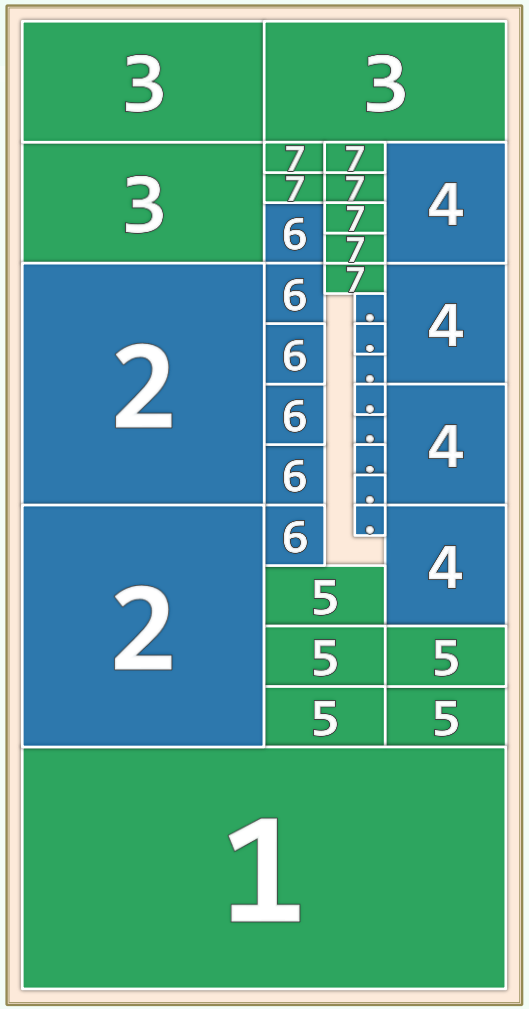
其中的循环，将迭代多少步？理论上讲，任意多次都有可能

❖ 迭代 k 步的概率：$P(s=k) = (1/2)^{k-1} \cdot (1-1/2) = 2^{-k}$

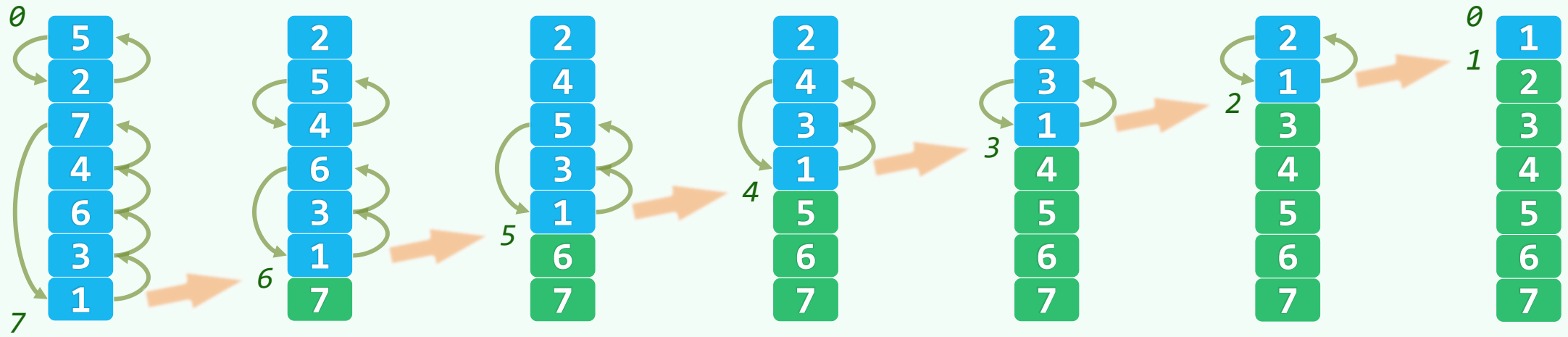
期望不过常数步：$\mathbb{E}(s) = \sum_{k=1}^{\infty} k \cdot 2^{-k} = 2$

❖ 一般地，若 `rand() & 1 == true` 的概率为 p ，则

$$\mathbb{E}_p(s) = \sum_{k=1}^{\infty} k \cdot p^{k-1} \cdot (1-p) = (1-p) \cdot \sum_{k=1}^{\infty} (p^k)' = (1-p) \cdot \Big(\sum_{k=1}^{\infty} p^k\Big)' = 1/(1-p)$$

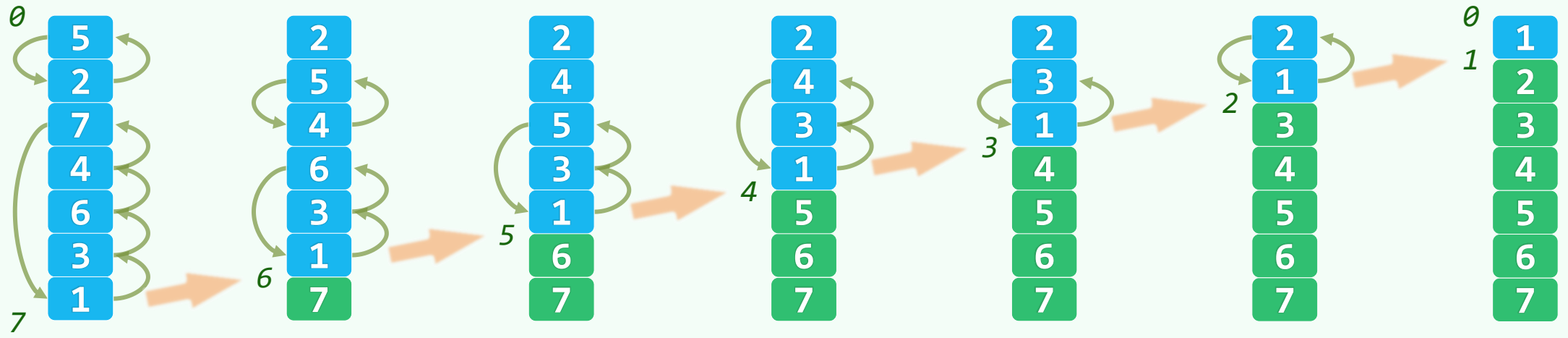


起泡排序：反复做起泡扫描




```
❖ void bubbleSort( int A[], int n ) {
    while ( 1 < n-- ) //做n-1趟起泡扫描：有序序列中，每一对相邻元素都有序
       for ( int i = 0; i < n; i++ ) //自前向后地逐对检视每对相邻元素
          if ( A[i] > A[i + 1] ) //一旦逆序，则
             swap( A[i], A[i + 1] ); //交换之
  } //bubblesort
```

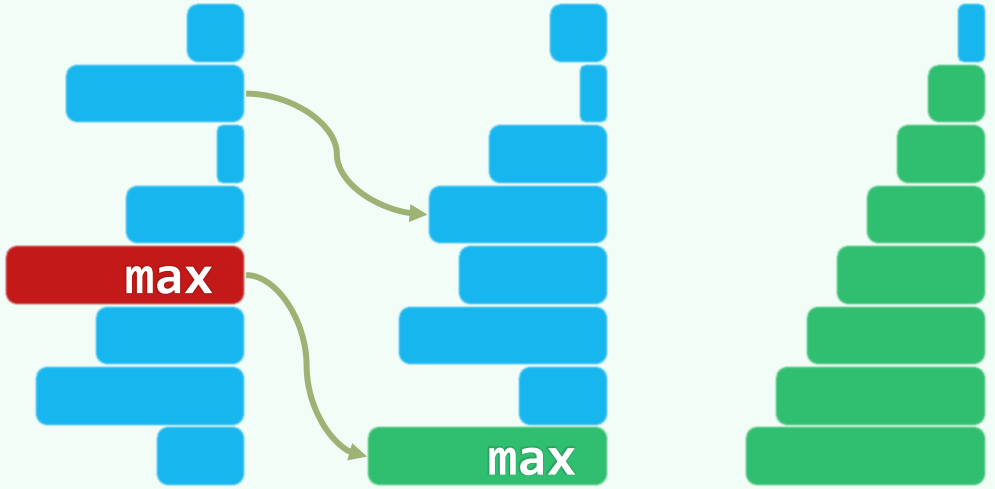
起泡排序：因不变，而正确



❖ 序列模式：未必有序的**前缀**[0,s) + 已然有序的**后缀**[s,n)

❖ 初始s = n；随后，每经过一趟起泡扫描

- [**实质元**] 前缀中的max，必然就位
- [**不变性**] 序列模式依然保持，且
- [**单调性**] 有序后缀都会增长



从正确的算法，到“好”的算法

- 语法正确，能够编译、链接
- 功能正确，能够处理：

  简单的输入、大规模的输入

  一般性的输入、退化的输入

  任意合法的输入

- 健壮： 能辨别不合法的输入并做适当处理

  而不致非正常退出
- 可读、易维护：结构化 + 命名 + 注释 + ...
- 效率： 速度尽可能快

  存储空间尽可能少

Algorithms + Data Structures = Programs

(Algorithms + Data Structures) x Efficiency = Computation

又快好省

1．绪论

01-A.计算

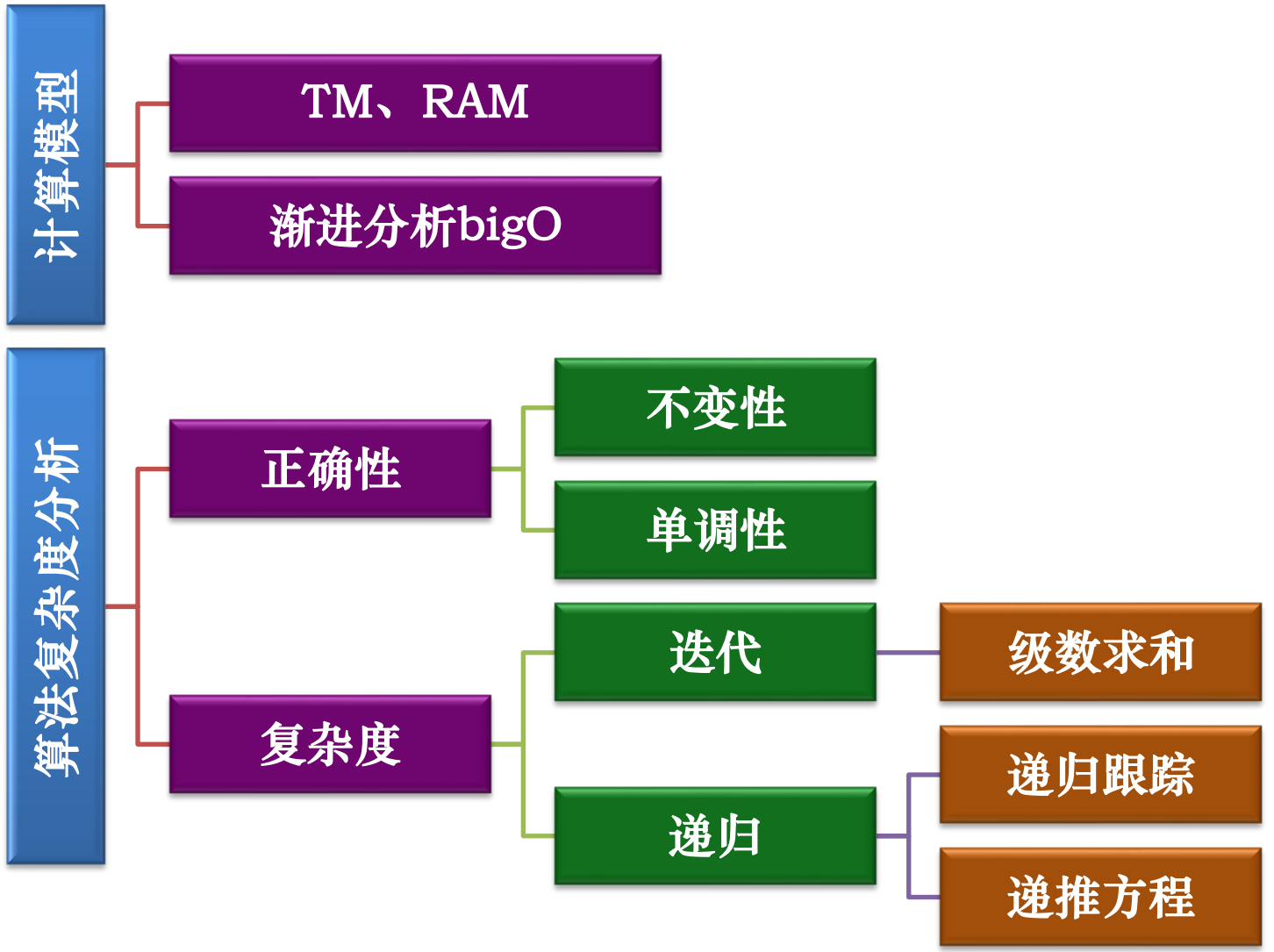
01-B.计算模型

01-C.渐进复杂度

01-D.复杂度分析

01-E.迭代与递归

01-F. 动态规划



01.Introduction

B-计算模型

绪论

计算模型：图灵机

邓俊辉
deng@tsinghua.edu.cn

特定算法 + 不同实例

❖ 令： $T_A(n)$ = 用算法A求解同一问题规模为n的实例，所需的计算成本

讨论特定算法A（及其对应的问题）时，可简记作T(n)

❖ 然而： 同等规模问题实例的计算成本不尽相同，甚至差异极大

❖ 例如： 任给平面上n个点，在它们定义的 $\binom{n}{3}$ 个三角形中，

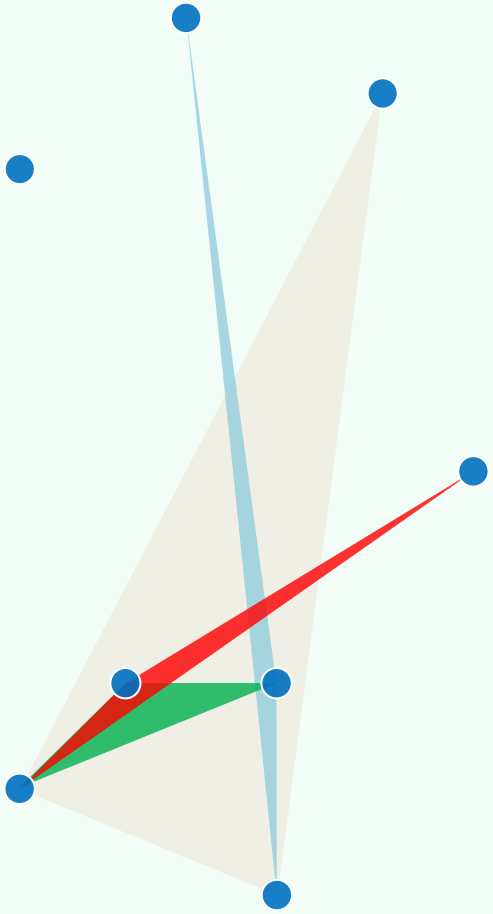
是否某一个的面积不超过5.0?

❖ 蛮力： 最坏情况下需枚举所有三角形；但运气好的话...

❖ 既然如此，又该如何定义T(n)呢?

❖ 稳妥起见，取 $T(n) = \max\{ T(P) \mid |P| = n \}$

亦即，在规模同为n的所有实例中，只关注最坏（成本最高）者

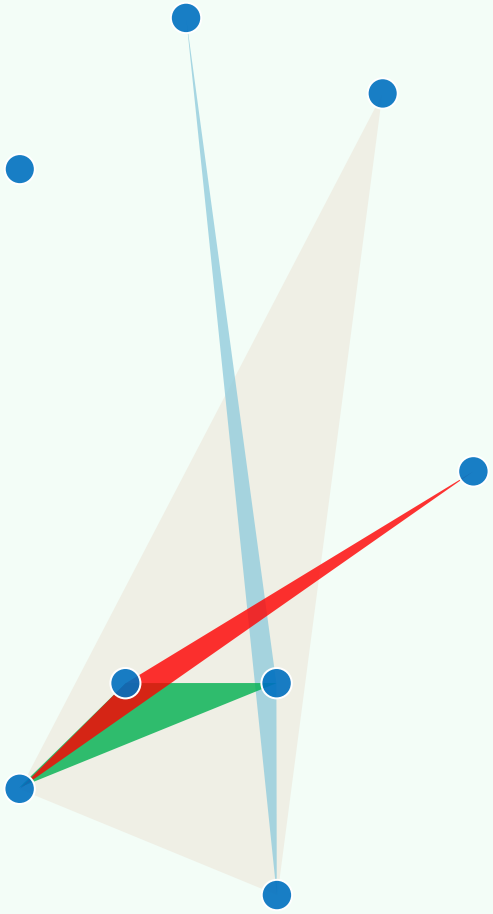


特定问题 + 不同算法

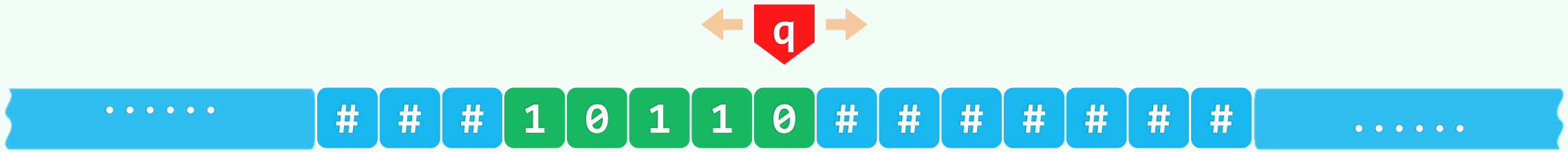
- 同一问题通常有多种算法，如何评判其优劣？

- 实验统计是最直接的方法，但并不足以准确反映算法的真正效率
  - 不同的算法，可能更适应于不同规模的输入
  - 不同的算法，可能更适应于不同类型的输入
  - 同一算法，可能由不同程序员、用不同程序语言、经不同编译器生成
  - 同一算法，可能实现并运行于不同的体系结构、操作系统...

- 为给出客观的评判，需要抽象出一个理想的平台或模型
  - 不再依赖于上述种种具体的因素
  - 从而直接而准确地描述、测量并评价算法



TM：构成部件



❖ Tape|大千世界

依次均匀地划分为单元（Cell）

各存有某一符号，初始均为'#'

❖ State|情绪

- 读写头（TM）总是处于某一状态
- 每经一拍，可按规则转向另一种状态

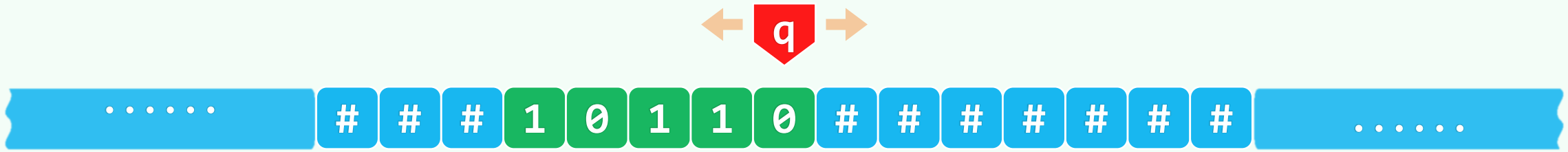
❖ Head|万物之灵

- 总是对准某一单元，并可读取或改写其中的字符
- 每经一拍，可左移或右移一个单元

❖ Alphabet|语言

- 使用有限种符号

TM：转换函数



- Transition Function:

  (q, c; d, L/R, p)

- 若当前状态为q，且当前字符为c，则
  - 将当前字符改写为d
  - 转向左/右侧邻格
  - 转入p状态

- Sequential Access：每次只能移动一步
- 特别地，一旦转入状态h，便随即停机
- 从启动至停机，所经历的节拍数目

  即可用以度量计算时间T(n)

- 亦等于Head累计的移动步数（或距离）

实例：Increase

❖ 功能：将二进制非负整数加一

❖ 原理：全'1'的后缀，翻转为全'0'

原最低位'0'或'#'翻转为'1'

❖ (<, 1; 0, L, <) //左行，1->0

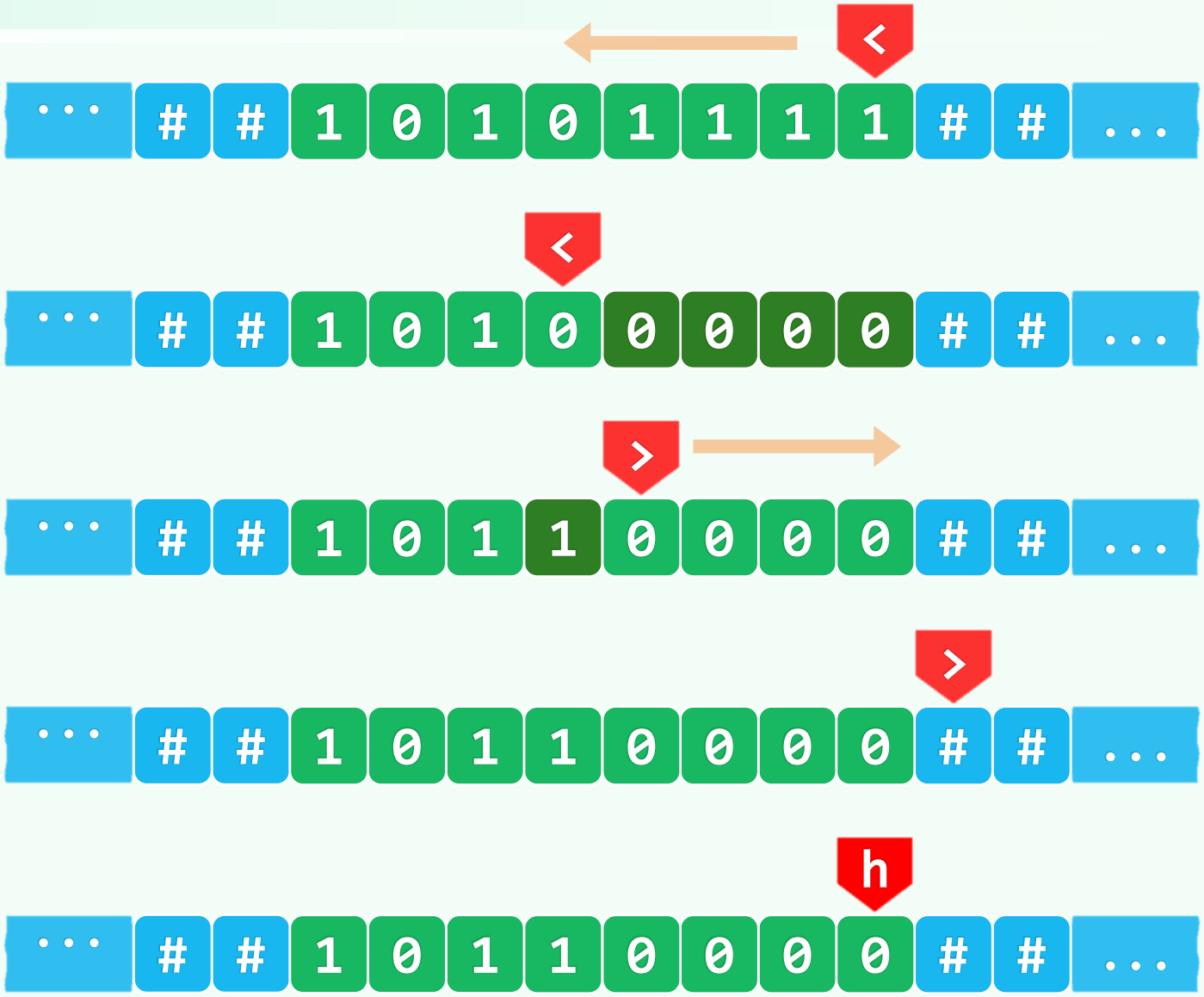
(<, 0; 1, R, >) //掉头，0->1

(<, #; 1, R, >) //可否省略?

(>, 0; 0, R, >) //右行

(>, #; #, L, h) //?

❖ T(n) = T(8) = 4+1+4+1 = 10



绪论

计算模型：RAM

邓俊辉
deng@tsinghua.edu.cn

Random Access Machine: 组成 + 语言



- **寄存器顺序编号，总数没有限制**

  //但愿如此

- **可通过编号直接访问任意寄存器**

  //call-by-rank

- **IR = Instruction Register**

  //指示当前正在执行哪条指令

- **每一基本操作仅需常数时间**

```
R[i] <- c                          GOTO #
R[i] <- R[j]
R[i] <- R[R[j]]                    IF R[i] = 0 GOTO #
R[R[i]] <- R[j]
R[i] <- R[j] + R[k]                IF R[i] > 0 GOTO #
R[i] <- R[j] - R[k]                STOP
```

Random Access Machine: 效率



- 与TM模型一样，RAM模型也是一般计算工具的简化与抽象

  使我们可以独立于具体的平台，对算法的效率做出可信度量与中肯评判

- 在这些模型中
  - 算法的运行时间 $\propto$ 算法需要执行的基本操作次数
  - T(n) = 算法为求解规模为n的问题，所需执行的基本操作次数

- 思考：在TM、RAM等模型中衡量算法效率，为何通常只需考查运行时间？空间呢？

实例：Ceiling Division：思路

❖ $\forall\ c \geq 0\ \text{ and }\ d > 0$, define

$$\lceil c/d \rceil\ =\ \min\{\ x \in \mathbb{Z}\ \big|\ c \leq d \cdot x\ \}$$

$$=\ \min\{\ x\ \big|\ c - 1 < d \cdot x\ \}$$

❖ **例如：** $\lceil 2/7 \rceil\ =\ 1$ $\lceil 2026/1014 \rceil\ =\ 2$

$\lceil 35/5 \rceil\ =\ 7$ $\lceil 2026/1013 \rceil\ =\ 2$

$\lceil 41/7 \rceil\ =\ 6$ $\lceil 2026/1012 \rceil\ =\ 3$

❖ **思路：反复地从 R[0] = c 中，减去 R[1] = d**

统计在下溢之前，所做减法的次数x

Step	IR	R[0]	R[1]	R[2]	R[3]
0	[0]	25	12	0	0

实例: Ceiling Division: 算法

```
[0] R[3] <- 1  //constant increment
[1] GOTO 4 //in case R[0] = c == 0
[2] R[2] <- R[2] + R[3]  //x++
[3] R[0] <- R[0] - R[1]  //c -= d
[4] IF R[0] > 0 GOTO 2   //if c > 0 goto 2
[5] R[0] <- R[2]  //else copy x to R[0] and
[6] STOP  //return R[0] = x = ⌈c/d⌉
```

❖ 时间成本 = 快照表的高度 = 各条指令累计的执行次数

T(25,12) = 1 + 1 + 3 + 3 + 4 + 1 + 1 = 14

Step	IR	R[0]	R[1]	R[2]	R[3]
0	[0]	25	12	0	0
1	[1]	^	^	^	1
2	[4]	^	^	^	^
3	[2]	^	^	^	^
4	[3]	^	^	1	^
5	[4]	13	^	^	^
6	[2]	^	^	^	^
7	[3]	^	^	2	^
8	[4]	1	^	^	^
9	[2]	^	^	^	^
10	[3]	^	^	3	^
11	[4]	-11	^	^	^
12	[5]	^	^	^	^
13	[6]	3	^	^	^

1．绪论

01-A.计算

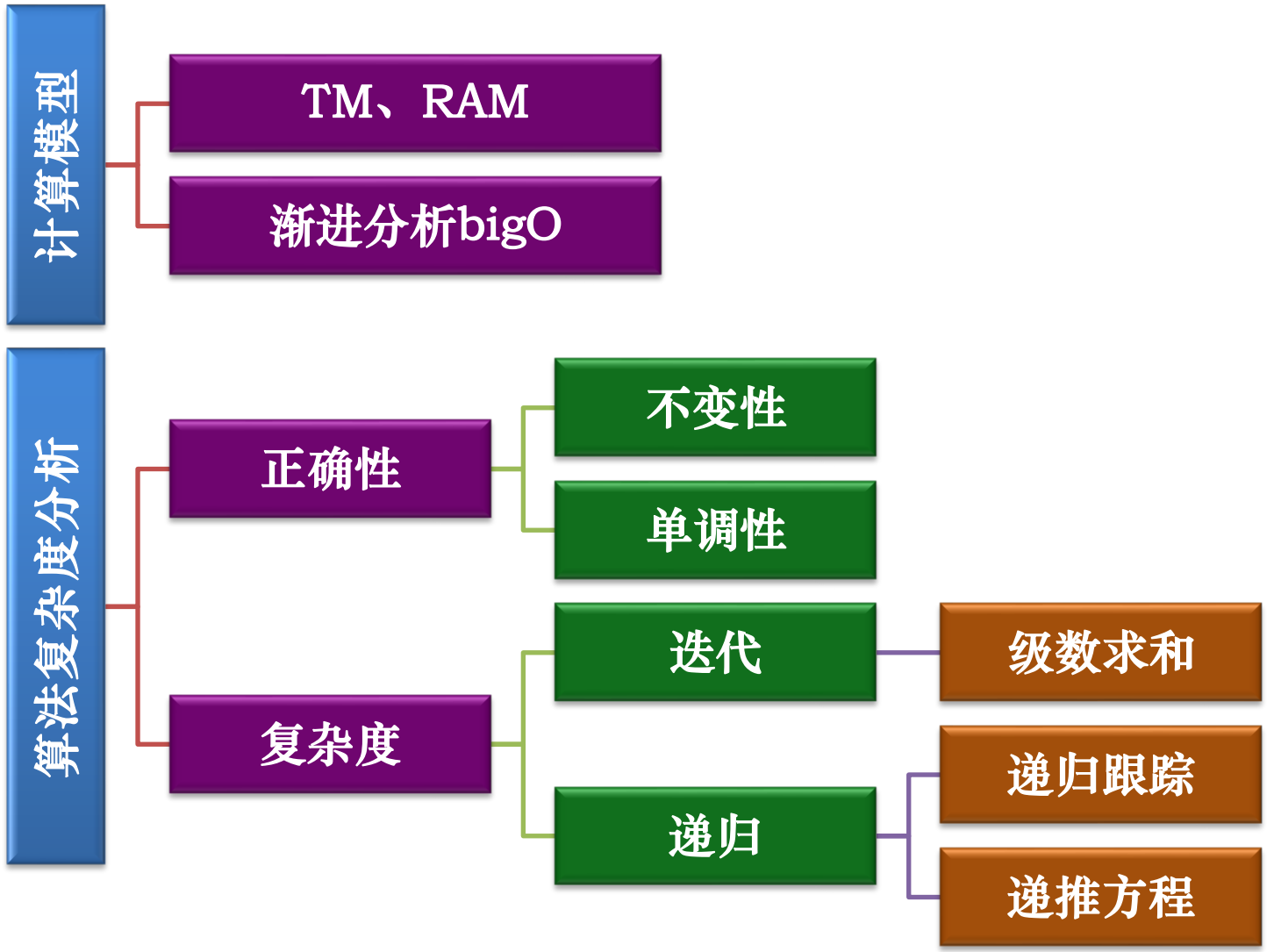
01-B.计算模型

01-C.渐进复杂度

01-D.复杂度分析

01-E.迭代与递归

01-F. 动态规划



一把尺子和三张表

- 大O符号（Big O notation）是用于描述函数渐进行为的数学符号，
  代表“order of ...”（...阶）

  确切地说，它是用另一个(更简单的)函数来描述一个函数**数量级的渐近上界**
  - 在数学中， Big O notation一般用来刻画被截断的无穷级数尤其是渐近级数的剩余项
  - 在计算机科学中，它在分析算法复杂性的方面非常有用。
- Big O 中，常数项、常系数、低次项皆可忽略，**仅关注最高阶次**

- 在这些模型中
  - 算法的运行时间 ∝ 算法需要执行的基本操作次数
  - T(n) = 算法为求解规模为n的问题，所需执行的基本操作次数

【表❶】算法复杂度

【表❷】级数求和

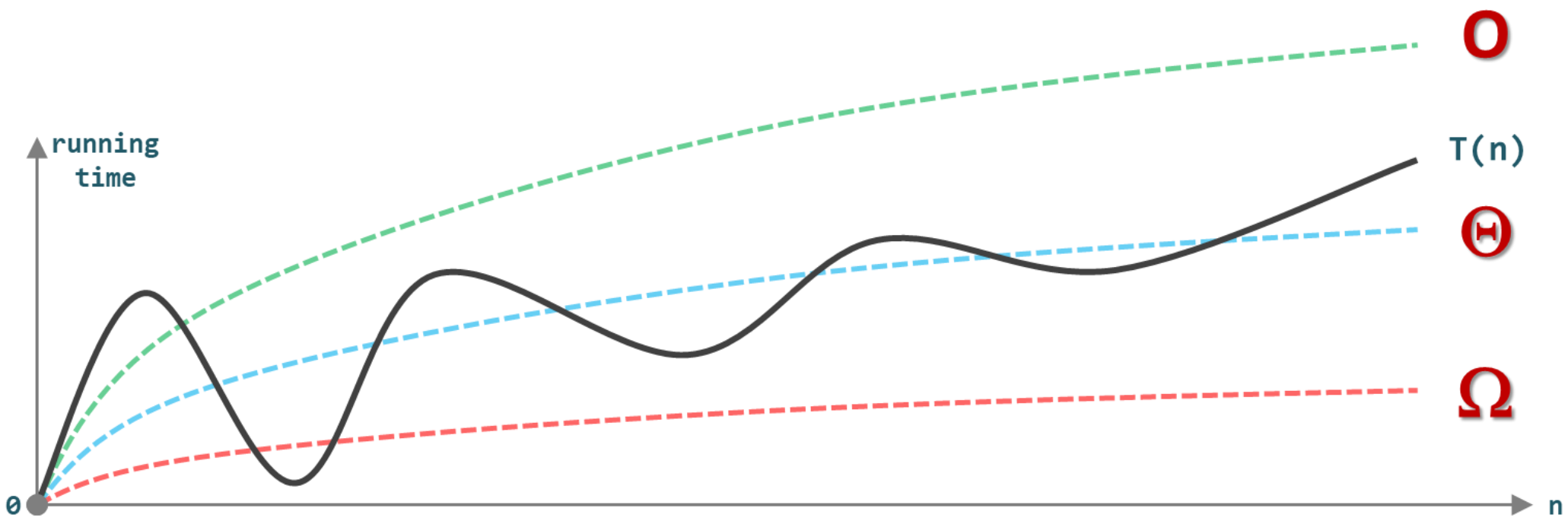
【表❸】递推方程

得到一把尺子：BIG O and others

$$T(n) = \mathcal{O}(f(n)) \quad \text{iff} \quad \exists\, c > 0 \quad \text{s.t.} \quad T(n) < c \cdot f(n) \quad \forall\, n \gg 2$$

$$T(n) = \Theta(f(n)) \quad iff \quad \exists\, c_1 > c_2 > 0 \quad s.t. \quad c_1 \cdot f(n) > T(n) > c_2 \cdot f(n) \quad \forall\, n \gg 2$$

$$T(n) = \Omega(f(n)) \quad iff \quad \exists\, c > 0 \quad s.t. \quad T(n) > c \cdot f(n) \quad \forall\, n \gg 2$$



【表❶】算法复杂度表（层次）

常数	O(1)	再好不过，但难得如此幸运	对数据结构的基本操作
	$O(\log^* n)$	在这个宇宙中，几乎就是常数	逆Ackermann函数
对数	O(logn)	与常数无限接近，且不难遇到	有序向量的二分查找 堆、词典的查询、插入与删除
线性	O(n)	努力目标，经常遇到	树、图的遍历
	$O(n\log^* n)$	几乎几乎几乎...就是线性	某些MST算法
	O(nloglogn)	非常非常非常...接近线性	某些三角剖分算法
	O(nlogn)	最常出现，但不见得最优	排序、EU、Huffman编码
平方	$O(n^2)$	所有输入对象两两组合	Dijkstra算法
立方	$O(n^3)$	不常见	矩阵乘法
多项式	$O(n^c)$	P问题 = 存在多项式算法的问题	
指数	$O(2^n)$	很多问题的平凡算法，再尽可能优化	
...		绝大多数问题，并不存在算法	

【表②】级数求和表（迭代）

算数级数：与末项平方同阶	$T(n) = 1 + 2 + ... + n$	$= O(n^2)$
幂级数：比幂次高出一阶	$T(n) = 1^2 + 2^2 + 3^2 + ... + n^2$	$= O(n^3)$
	$T(n) = 1^3 + 2^3 + 3^3 + ... + n^3$	$= O(n^4)$
	$T(n) = 1^4 + 2^4 + 3^4 + ... + n^4$	$= O(n^5)$
几何级数：与末项同阶	$T(n) = 1 + 2 + 4 + ... + 2^n$	$= O(2^n)$
收敛级数：	$T(n) = 1/1/2 + 1/2/3 + 1/3/4 + ... + 1/(n-1)/n$	$= O(1)$
	$T(n) = 1 + 1/2^2 + ... + 1/n^2 < 1 + 1/2^2 + ...$	$= O(1)$
调和级数：	$h(n) = 1 + 1/2 + 1/3 + ... + 1/n$	$= \Theta(\log n)$
对数级数：	$\log 1 + \log 2 + \log 3 + ... + \log n = \log(n!)$	$= \Theta(n \log n)$

【表③】递推方程表（递归）

Recurrence	Solution	Examples
T(n) = T(n-1) + O(1)	O(n)	向量求和之线性递归版
T(n) = T(n-1) + O(n)	$O(n^2)$	列表起泡排序之线性递归版
T(n) = 2*T(n-1) + 1	$O(2^n)$	Hanoi塔、Fibonacci数
T(n) = 2*T(n-1) + n	$O(2^n)$	
T(n) = T(n/2) + 1	O(logn)	向量的二分查找
T(n) = T(n/2) + n	O(n)	列表的二分查找
T(n) = 2*T(n/2) + 1	O(n)	向量求和之二分递归版
T(n) = 2*T(n/2) + n	O(nlogn)	归并排序

01.Introduction

C-渐进复杂度

绪论

渐近复杂度：大O记号

邓俊辉
deng@tsinghua.edu.cn

❖ 回到原先的问题：

随着问题规模的增长，计算成本如何增长?

❖ 这里更关心：

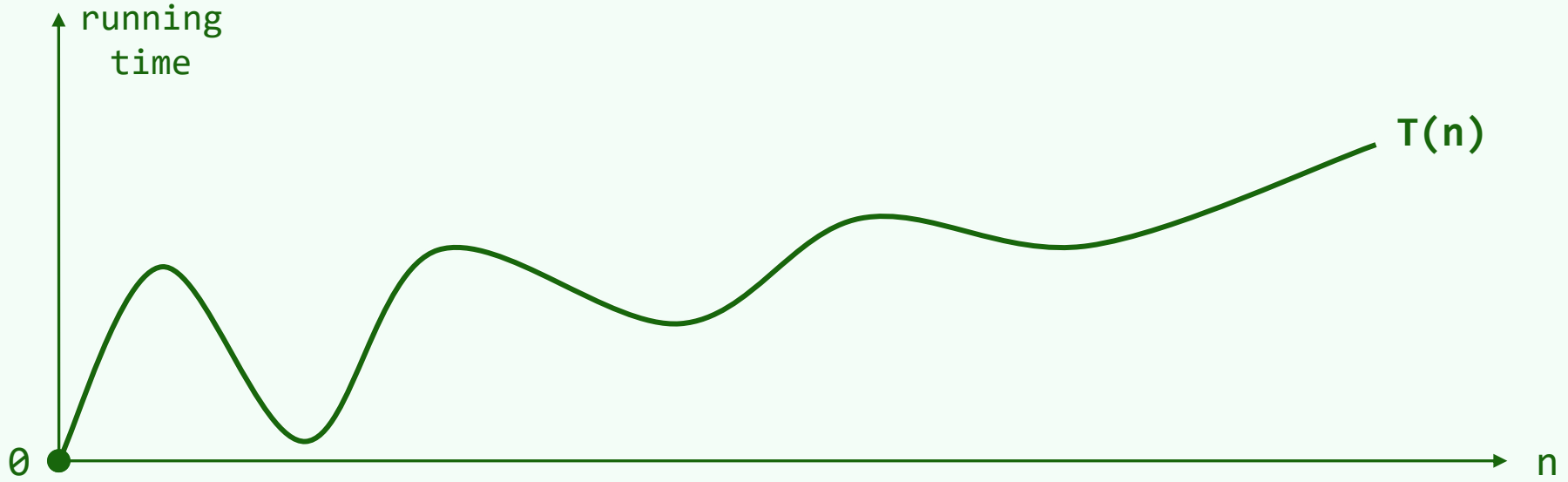
问题规模足够大之后，计算成本的增长趋势

❖ 当输入规模 n ≫ 2 后，算法

需执行的基本操作次数 T(n) = ?

❖ 如欲更为精确地估计，还可考查

需占用的存储单元数 S(n) = ?



Big O notation: 来历

- 大O符号（Big O notation）是由德国数论学家保罗·巴赫曼（Paul Bachmann）在其1894年的著作《解析数论》（Analytische Zahlentheorie）首先引入的。
  大O符号则是在另一位德国数论学家艾德蒙·朗道(Edmund Landau)的著作中才推广的，因此它有时又称为朗道符号(Landau symbol)。
- 代表“order of ...”（...阶）的大O，最初是一个大写的希腊字母'Ο'（Omicron），现今用的是英文大写字母'O'，但从来不是阿拉伯数字'0'。
- 大O符号（Big O notation）是用于描述函数渐进行为的数学符号。
  更确切地说，它是用另一个（通常更简单的）函数来描述一个函数**数量级的渐近上界**。
  - 在数学中， Big O notation一般用来刻画被截断的无穷级数尤其是渐近级数的剩余项；
    在计算机科学中，它在分析算法复杂性的方面非常有用。

Big O notation：无穷大渐进

- 大O符号在分析算法效率的时候非常有用。

  例如：解决一个规模为 n 的问题所花费的时间（或所需步骤的数目）被求得：

  $T(n) = 4n^2 - 2n + 2$。

  - 当 n 增大时，n^2项将开始占主导地位，而其他各项可以被忽略。

    当 n=500，$4n^2$项是2n项的1000倍，

    因此在大多数场合下，**省略低次幂**对表达式的值的影响将是可以忽略不计的。

- 有时，与任一其他级的表达式比较，**n^2项的系数**也可以是无关紧要的

  - 例如$T(n) = 1,000,000\ n^2$，

    一旦 n 增长到大于1,000,000，后者就会一直超越前者($T(1,000,000) = 1,000,000^3$ 。

    这样，大O符号就记下剩余的部分，写作：$T(n) \in O(n^2)$ 或者$T(n)=O(n^2)$;

    就说该算法具有**平方阶**的时间复杂度。

Big O 中，常数项、常系数、低次项皆可忽略，仅关注最高阶次

次数	算法A（8n+9）	算法B（2n^2）	算法C（2n^2+3n+1）
n=1	17	2	6
n=2	25	8	15
n=5	49	50	66
n=10	89	200	231
n=100	809	20 000	20 301
n=1,000	8 009	2 000 000	2 003 001
n=10,000	80 009	200 000 000	200 030 001
n=100,000	800 009	20 000 000 000	20 000 300 001
n=1,000,000	8 000 009	2 000 000 000 000	2 000 003 000 001

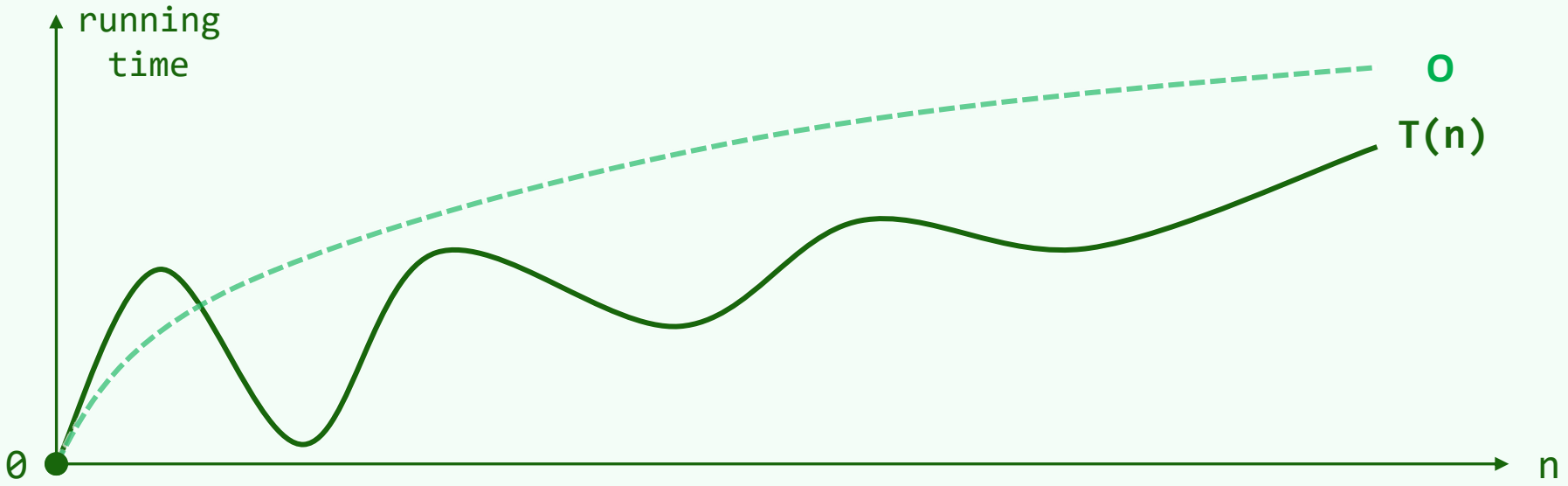
Big-O notation

❖ **Paul Bachmann, 1894:** $T(n) = \mathcal{O}(f(n))$ **iff** $\exists\, c > 0$ **s.t.** $T(n) \leq c \cdot f(n) \quad \forall\, n \gg 2$

$$\sqrt{5n \cdot [3n \cdot (n+2)+4]+6} \;<\; \sqrt{5n \cdot [6n^2+4]+6} \;<\; \sqrt{35n^3+6} \;<\; 6 \cdot n^{1.5} \;=\; \mathcal{O}(n^{1.5})$$

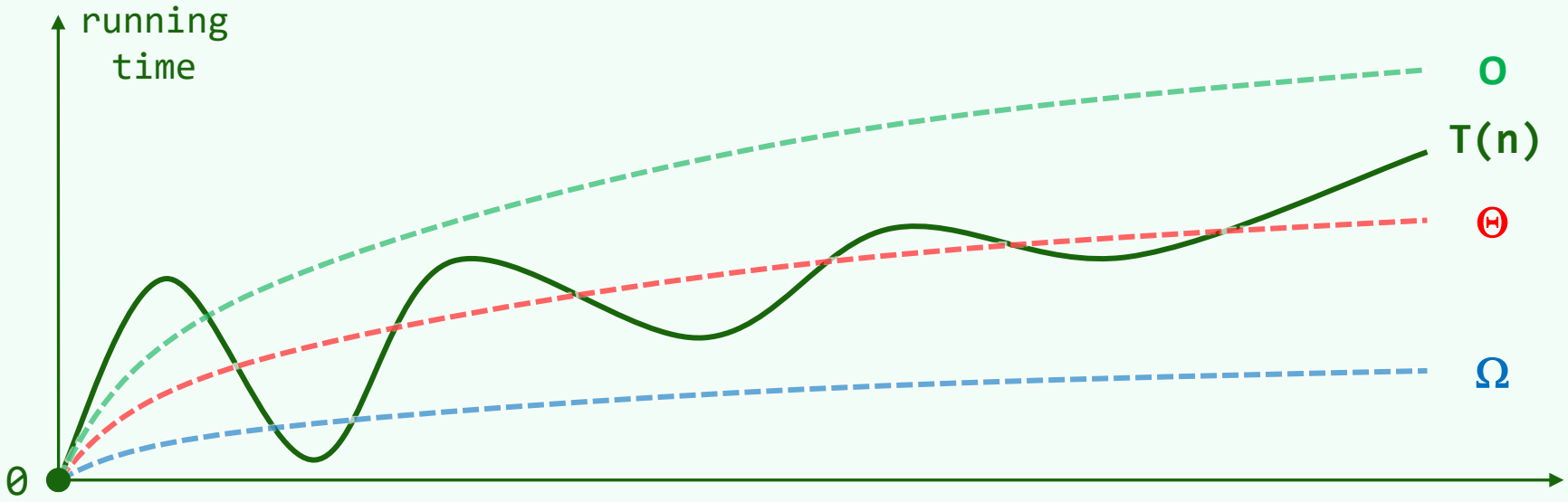
❖ **与T(n)相比，f(n)在形式上更为简洁，但依然反映前者的增长趋势**

$$\mathcal{O}(f(n)) \;=\; \mathcal{O}(c \cdot f(n)) \qquad\qquad \mathcal{O}(n^a \pm n^b) \;=\; \mathcal{O}(n^a),\; a \geq b > 0$$



其它记号

- $T(n) = \Omega(f(n))$ **iff** $\exists\, c > 0$ **s.t.** $T(n) \geq c \cdot f(n) \quad \forall\, n \gg 2$
- $T(n) = \Theta(f(n))$ **iff** $\exists\, c_1 \geq c_2 > 0$ **s.t.** $c_1 \cdot f(n) \geq T(n) \geq c_2 \cdot f(n) \quad \forall\, n \gg 2$



空间复杂度

多快好省

❖ Storage Complexity:

算法执行过程中，会消耗多少空间

❖ 好消息：各种渐近记号，可以直接借用

❖ 约定：输入本身所占空间，通常不计

因为：属于固有消耗，与算法优劣无关

❖ 其余的所有，都需计入：

转储、缓冲、中继、索引、变换...

❖ 往往：不必重点考查空间复杂度

因为：无论是在TM还是RAM模型中

- 一次基本操作至多消耗$O(1)$空间
- 时间复杂度就是空间复杂度天然的一个上界

❖ 如何才能达到紧界？

- TM：Head一旦出发，便义无反顾、誓不回头
- RAM：每次都须开辟处女地，绝不故地重游

❖ 当然，若非常在乎空间效率，则需另当别论

绪论

渐近复杂度：多项式与可解

邓俊辉
deng@tsinghua.edu.cn

O(1): constant

不含转向的语句，必然是顺序执行的，即时间复杂度为O(1)

- 常数：2 = 2026 = 2026 × 2026 = O(1)　　//含RAM的所有基本操作，甚至

  2026^{2026} = O(1)

- 从渐近的角度来看，再大的常数，也要小于递增的变数　　//尽管实际并非如此

- [General Twin Prime Conjecture, de Polignac 1849]

  For every natural number k, there are infinitely many prime pairs p and q such that p – q = 2k

- [Yitang Zhang, April 2013]　　k ≤ 35,000,000

  [Terence Tao, May 2013]　　k ≤ 6,500,000

  [Polymath Project, April 2014]　k ≤ 123

  ...

O(1): constant

- 这类算法的效率，自然已属最高了 //总不能奢望不劳而获吧
- 一定不含循环?

```
for ( i = 0; i < n; i += n/2026 + 1 );          //常数
for ( i = 1; i < n; i = 1 << i );               //log*n，几乎常数
```

- 一定不含分支转向?

```
if ( (n+m)*(n+m) < 4*n*m ) goto UNREACHABLE;    //不考虑溢出
```

- 一定不能有（递归）调用?

```
if ( 2 == (n*n) % 5 ) neverCalled(n);           //Qudratic Residue
```

- 空间复杂度必然也是O(1)：仅需常数辅助空间的算法，称作就地算法|In-place algorithm

$\mathcal{O}(\log^c n)$: poly-log

❖ **对数** $\mathcal{O}(\log n)$：　$\ln n$　$\lg n$　$\log_{100} n$　$\log_{2026} n$　//为何不注明底数?

❖ **常底数无所谓**：$\forall\, a, b > 1,\ \log_a n = \boxed{\log_a b} \cdot \log_b n = \Theta(\log_b n)$

常数次幂无所谓：$\forall\, c > 0,\ \log n^c = c \cdot \log n = \Theta(\log n)$

❖ **对数多项式**：$123 \cdot \log^{321}(n^{2026}) + 2026 \cdot \log^{123}(n^{321}) = \mathcal{O}(\log^{321} n)$

❖ **这类算法非常有效，复杂度无限接近于常数**：$\forall\, c > 0,\ \log n = \mathcal{O}(n^c)$

O(log*n)

- 迭代对数（iterated logarithm）也称为重复对数，一般会用log*n来表示。
- log*n是一个增加非常慢的数学函数，比对数要慢很多，可以视为近似常数。
- 实数的迭代对数是指需对实数连续进行几次对数运算后，其结果才会小于等于1。

  最简单的定义如下递归函数：

$$\log^* n := \begin{cases} 0 & \text{if } n \leq 1; \\ 1 + \log^*(\log n) & \text{if } n > 1 \end{cases}$$

- 在计算机科学中，lg* 常用来表示实数

  可以进行几次以2为底的对数运算

x	lg* x
$(-\infty, 1]$	0
$(1, 2]$	1
$(2, 4]$	2
$(4, 16]$	3
$(16, 65536]$	4
$(65536, 2^{65536}]$	5

$O(n^c)$: polynomial

多项式复杂度，是可求解的复杂度，是可以被接受的。

❖ **多项式：** $a_k \cdot n^k + a_{k-1} \cdot n^{k-1} + \cdots + a_2 \cdot n^2 + a_1 \cdot n + a_0 = \mathcal{O}(n^k), \quad a_k > 0$

$$100 \cdot n + 2026 = \mathcal{O}(n) \qquad \sqrt{23 \cdot n - 472} \times \sqrt{101 \cdot n + 2026} = \mathcal{O}(n)$$

$$(100 \cdot n - 532) \cdot (20 \cdot n^2 - 445 \cdot n + 2026) = \mathcal{O}(n^3) \qquad (2026 \cdot n^2 - 129)/(1911 \cdot n - 37) = \mathcal{O}(n)$$

$$\sqrt[5]{4 \cdot n^4 \cdot \log^4 n + \sqrt[3]{43 \cdot n^{34} \cdot \log^{43} n + \sqrt[3]{432 \cdot n^{234} \cdot \log^{432} n + \sqrt[3]{4321 \cdot n^{1234} \cdot \log^{4321} n + n^{2025}}}}} = \mathcal{O}(n^{15})$$

❖ **线性|linear function：所有O(n)类函数**

❖ **从O(n)到O(n²)，是课程主要覆盖的范围：低了太稀有，多以成为经典；高了，基本上没法实用**

❖ **这类算法的效率通常认为已可令人满意，然而...这个标准是否太低了？** //P难度!

绪论

渐近复杂度：指数与难解

邓俊辉
deng@tsinghua.edu.cn

$\mathcal{O}(2^n)$: exponential

❖ **指数：** $T(n) = \mathcal{O}(a^n),\ \ a > 1$

$$\because\ \ e^n = 1 + n + n^2/2! + n^3/3! + n^4/4! + \ldots$$

$$\therefore\ \ \forall\, c > 0, a > 1,\ n^c = \mathcal{O}(a^n)$$

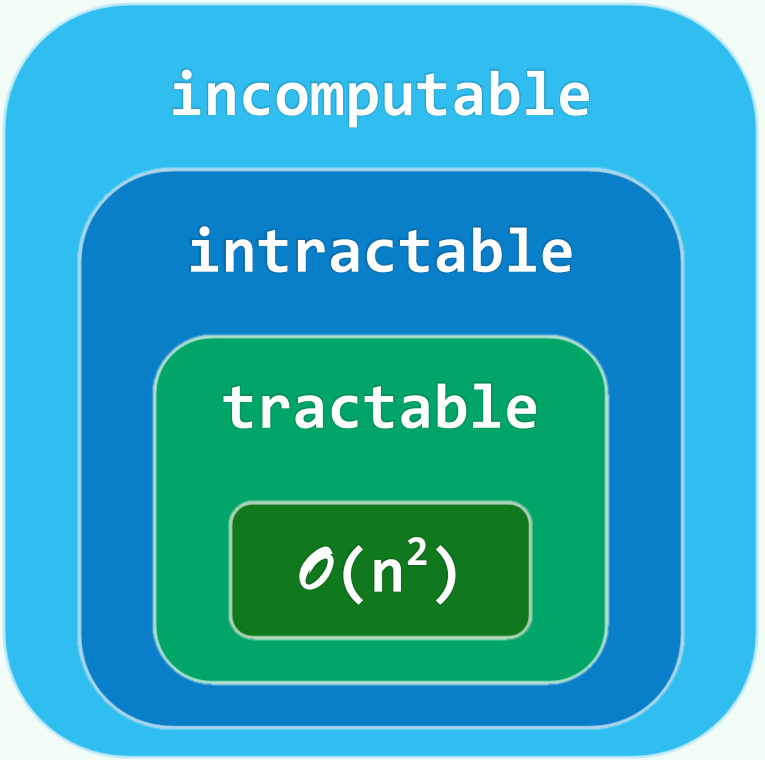
❖ **比如：** $n^{2026} = \mathcal{O}(1.0001^n)$

❖ **这类算法的计算成本增长极快，通常被认为不可忍受**

❖ **从 $\mathcal{O}(n^c)$ 到 $\mathcal{O}(2^n)$，是从有效算法到无效算法的分水岭**

❖ **$\mathcal{O}(2^n)$ 算法往往显而易见，然而设计出 $\mathcal{O}(n^c)$ 算法却极其不易，有时甚至注定是徒劳无功**

❖ **更糟糕的是，这类问题要远比我们想象的多得多...**



SubsetSum：问题

$$\forall\, S = \{ a_1, a_2, \ldots, a_n \} \subset \mathbb{Z}^+$$

$$0 \le t \le s = \sum_{k=1}^{n} a_k$$

$$\exists\, T \subseteq S \quad \text{s.t.} \quad \sum_{a \in T} a = t \ ?$$

❖ 从那堆石头里，曹冲真能挑出几块 //s

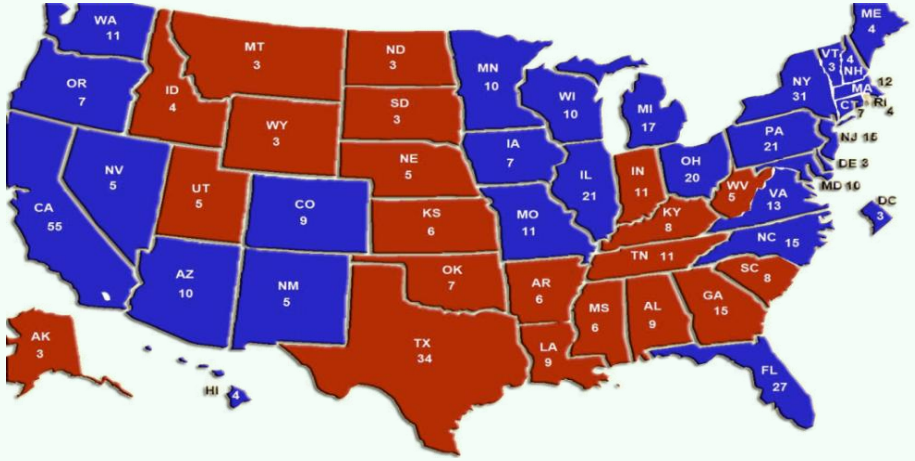
恰好与大象一样重？ //t

❖ 选举人团投票制：51个选区，共538票 //n，2t

若仅两位候选人，会否恰好各得269票？ //t

可视作SubsetSum的特例：$s = \sum_{k=1}^{n} a_k = 2t$

55	California	11	Indiana	7	Connecticut	4	Idaho
34	Texas	11	Missouri	7	Iowa	4	Maine
31	New York	11	Tennessee	7	Oklahoma	4	New Hampshire
27	Florida	11	Washington	7	Oregon	4	Rhode Island
21	Illinois	10	Arizona	6	Arkansas	3	Alaska
21	Pennsylvania	10	Maryland	6	Kansas	3	Delaware
20	Ohio	10	Minnesota	6	Mississippi	3	D. C.
17	Michigan	10	Wisconsin	5	Nebraska	3	Montana
15	Georgia	9	Alabama	5	Nevada	3	North Dakota
15	New Jersey	9	Colorado	5	New Mexico	3	South Dakota
15	North Carolina	9	Louisiana	5	Utah	3	Vermont
13	Virginia	8	Kentucky	5	West Virginia	3	Wyoming
12	Massachusetts	8	South Carolina	4	Hawaii	538 = Σ	



SubsetSum: 算法程序

❖ 直觉上，似乎并不难：

逐一枚举S的每一子集，分别统计总和并核对

❖ sSum(S = {a_1, a_2, ..., a_n}, t)

```
if ( t == 0 ) return true;

if ( n == 0 ) return false;

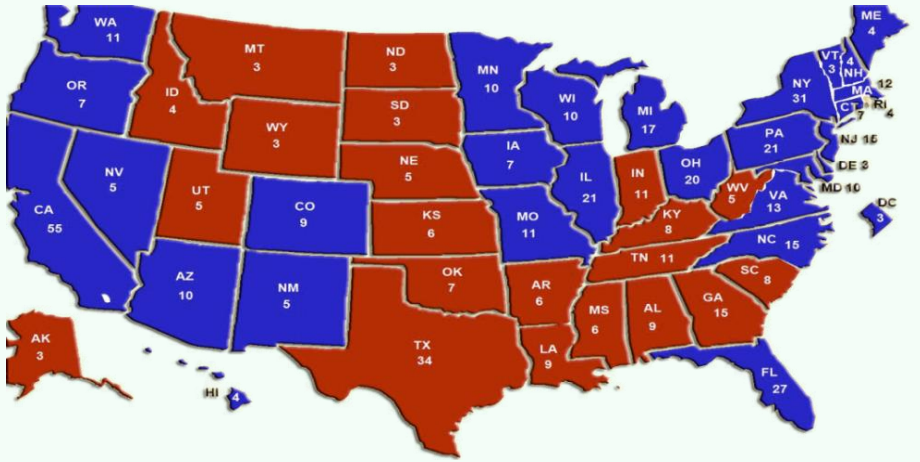
S = S\{a_n}; //classification

return sSum(S, t-a_n) || sSum(S, t);
```

❖ 最坏情况下，总需检视每一个子集，然而...

$$\left|2^S\right| = 2^{|S|} = 2^n$$

55	California	11	Indiana	7	Connecticut	4	Idaho
34	Texas	11	Missouri	7	Iowa	4	Maine
31	New York	11	Tennessee	7	Oklahoma	4	New Hampshire
27	Florida	11	Washington	7	Oregon	4	Rhode Island
21	Illinois	10	Arizona	6	Arkansas	3	Alaska
21	Pennsylvania	10	Maryland	6	Kansas	3	Delaware
20	Ohio	10	Minnesota	6	Mississippi	3	D. C.
17	Michigan	10	Wisconsin	5	Nebraska	3	Montana
15	Georgia	9	Alabama	5	Nevada	3	North Dakota
15	New Jersey	9	Colorado	5	New Mexico	3	South Dakota
15	North Carolina	9	Louisiana	5	Utah	3	Vermont
13	Virginia	8	Kentucky	5	West Virginia	3	Wyoming
12	Massachusetts	8	South Carolina	4	Hawaii	538 = Σ	



SubsetSum: NPC

- 故严格地讲，这类方法

  只能算作程序，而非算法

- 还是直觉：应该有更好的办法吧?

  比如...转化为...背包问题...?

  很遗憾，这里对于整数的取值范围未作任何假定

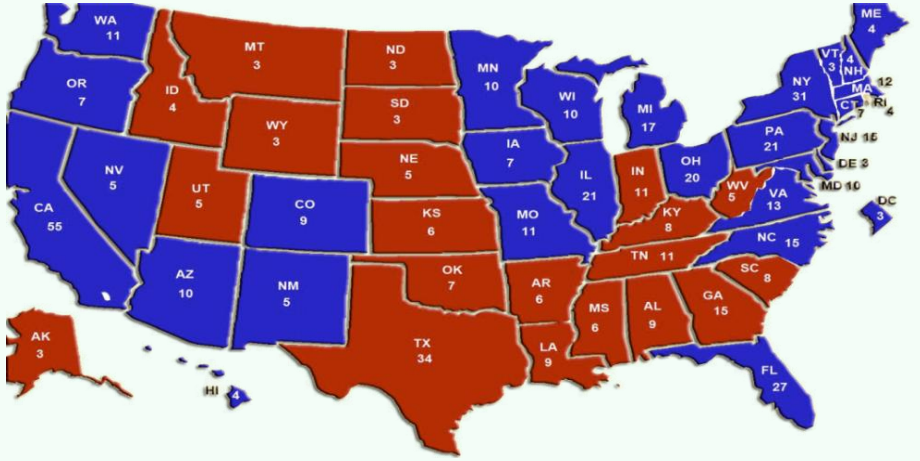
- SubsetSum is NP-complete —— 什么意思?

- 意即：就目前的计算模型而言

  不存在可在多项式时间内解决此问题的算法

  上述的直觉算法，居然就是最优的

55	California	11	Indiana	7	Connecticut	4	Idaho
34	Texas	11	Missouri	7	Iowa	4	Maine
31	New York	11	Tennessee	7	Oklahoma	4	New Hampshire
27	Florida	11	Washington	7	Oregon	4	Rhode Island
21	Illinois	10	Arizona	6	Arkansas	3	Alaska
21	Pennsylvania	10	Maryland	6	Kansas	3	Delaware
20	Ohio	10	Minnesota	6	Mississippi	3	D. C.
17	Michigan	10	Wisconsin	5	Nebraska	3	Montana
15	Georgia	9	Alabama	5	Nevada	3	North Dakota
15	New Jersey	9	Colorado	5	New Mexico	3	South Dakota
15	North Carolina	9	Louisiana	5	Utah	3	Vermont
13	Virginia	8	Kentucky	5	West Virginia	3	Wyoming
12	Massachusetts	8	South Carolina	4	Hawaii	538 = Σ	



渐近复杂度层次

常数	$O(1)$	再好不过，但难得如此幸运	对数据结构的基本操作
	$O(\log^* n)$	在这个宇宙中，几乎就是常数	没有最小，只有更小，比如逆Ackermann函数
对数	$O(\log n)$	与常数无限接近，且不难遇到	有序向量的二分查找；堆、词典的查询、插入与删除
线性	$O(n)$	努力目标，经常遇到	树、图的遍历
	$O(n\log^* n)$	几乎几乎几乎...就是线性	某些MST算法
	$O(n\log\log n)$	非常非常非常...接近线性	某些三角剖分算法
	$O(n\log n)$	最常出现，但不见得最优	排序、EU、Huffman编码
平方	$O(n^2)$	所有输入对象两两组合	Dijkstra算法
立方	$O(n^3)$	不常见	矩阵乘法
多项式	$O(n^c)$	P问题 = 存在多项式算法的问题	
指数	$O(2^n)$	很多问题的平凡算法，再尽可能优化	
...		绝大多数问题，**并不**存在算法	

【表❶】算法复杂度表（层次）

常数	O(1)	再好不过，但难得如此幸运	对数据结构的基本操作
	$O(\log^* n)$	在这个宇宙中，几乎就是常数	逆Ackermann函数
对数	O(logn)	与常数无限接近，且不难遇到	有序向量的二分查找 堆、词典的查询、插入与删除
线性	O(n)	努力目标，经常遇到	树、图的遍历
	$O(n\log^* n)$	几乎几乎几乎...就是线性	某些MST算法
	O(nloglogn)	非常非常非常...接近线性	某些三角剖分算法
	O(nlogn)	最常出现，但不见得最优	排序、EU、Huffman编码
平方	$O(n^2)$	所有输入对象两两组合	Dijkstra算法
立方	$O(n^3)$	不常见	矩阵乘法
多项式	$O(n^c)$	P问题 = 存在多项式算法的问题	
指数	$O(2^n)$	很多问题的平凡算法，再尽可能优化	
...		绝大多数问题，并不存在算法	

1．绪论

01-A.计算

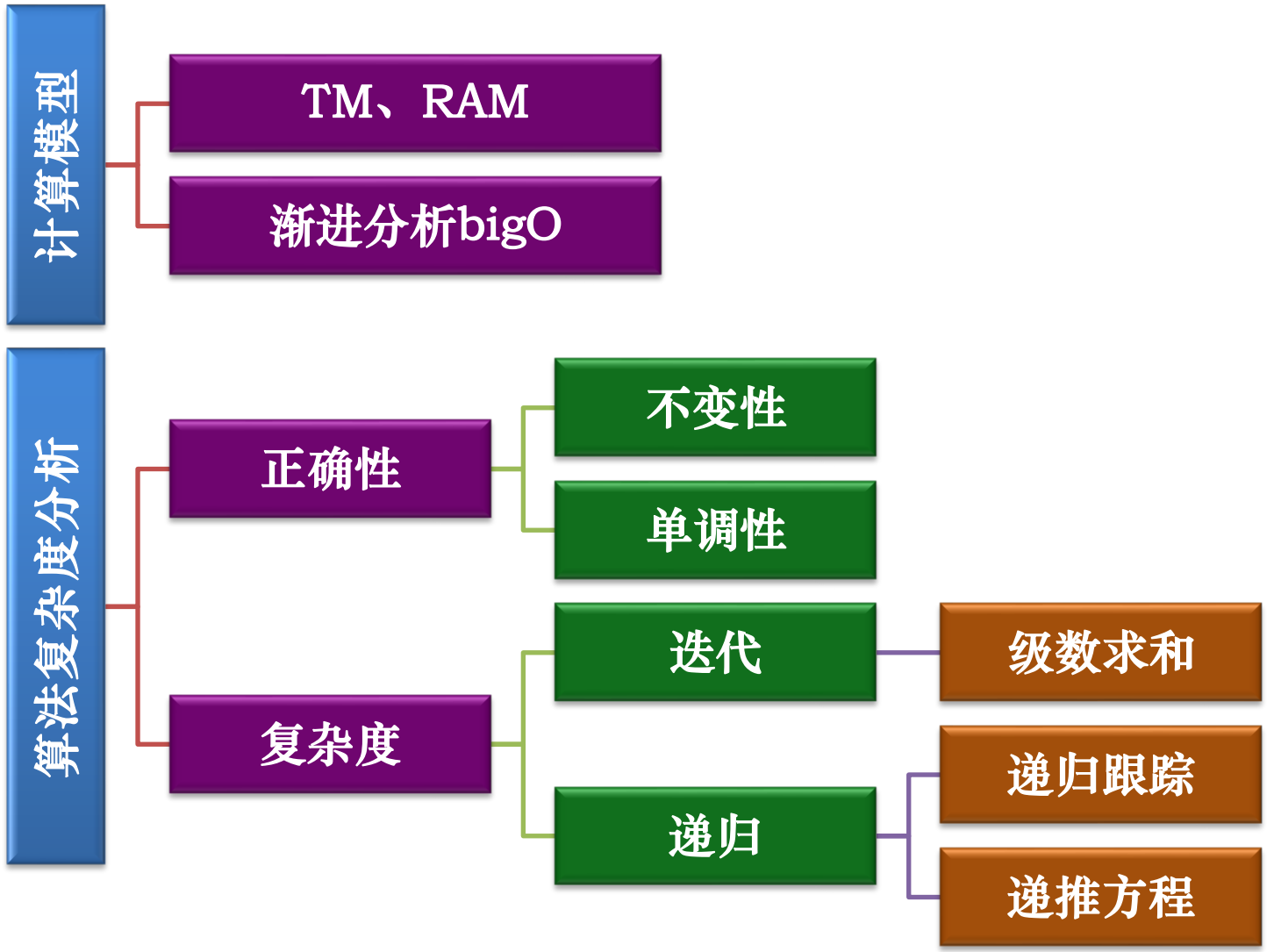
01-B.计算模型

01-C.渐进复杂度

01-D.复杂度分析

01-E.迭代与递归

01-F. 动态规划



01.Introduction

D-复杂度分析

绪论

复杂度分析：封底估算

定量估算

邓俊辉
deng@tsinghua.edu.cn

Back-Of-The-Envelope Calculation

- 赤道周长 ≈ 787 x 360/7.2 = 787 x 50 = 39,350 km

- 1天 = 24小时 x 60分 x 60秒 ≈ 25 x 4000 = 10^5秒

  1世 = 1代 = 30年 x 365天 = 10^4天 = 10^9秒

  1生 ≈ 1世纪 = 100年 x 365天 = 3 x 10^4天 = 3 x 10^9秒

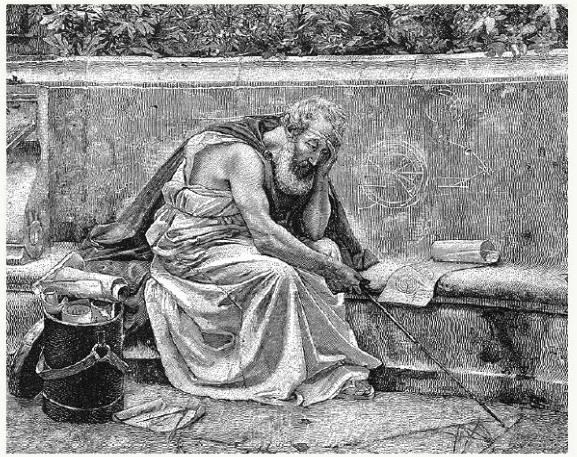
  三生三世 ≈ 300年 = 10^{10}秒

- 为祖国健康工作五十年 = 1,600,000,000秒

  宇宙大爆炸至今 = 4 x 10^{17}秒



Enrico Fermi
(1901 - 1954)



Eratosthenes
(276 ~ 194 B.C.)

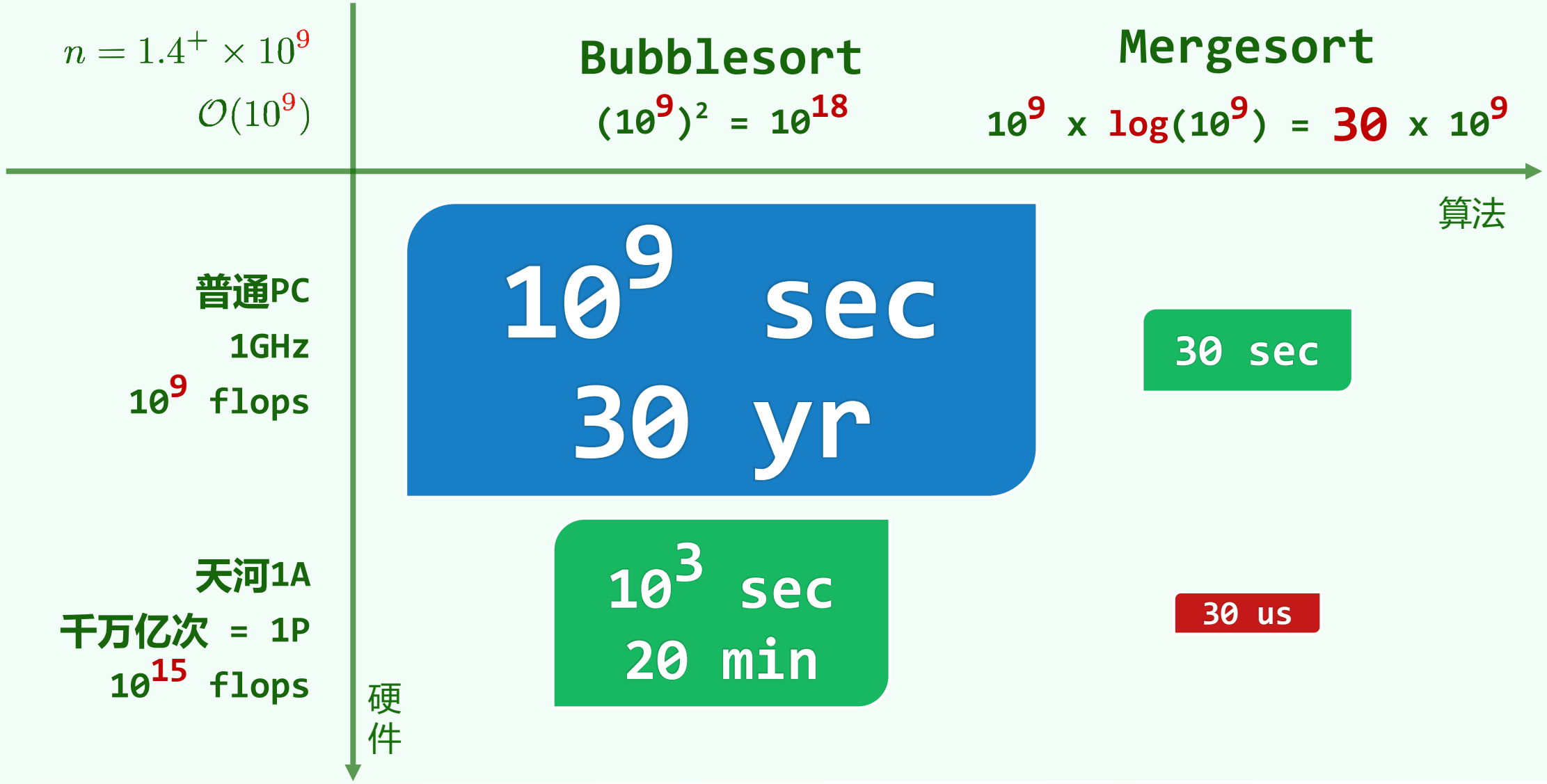
渐近时间复杂度

n	10^3	10^6	10^9	10^12	10^15	10^18	10^21
$\mathcal{O}(\log^* n)$	3	<= 4 (if n < 2^(2^(2^(2^2))) = 10^{19660})					
$\mathcal{O}(\log\log n)$	3	< 10 (if n < 10^{300})					
$\mathcal{O}(\log n)$	10	20	30	40	50	60	70
$\mathcal{O}(\sqrt{n})$	30	10^3	3*10^4	10^6	3*10^7	10^9	3*10^10
$\mathcal{O}(n)$	10^3	10^6	10^9	10^12	10^15	10^18	10^21
$\mathcal{O}(n \cdot \log n)$	10^4	2*10^7	3*10^10	4*10^13	5*10^16	6*10^19	7*10^22
$\mathcal{O}(n \cdot \sqrt{n})$	3*10^4	10*9	3*10^13	10^18	3*10^22	10*27	3*10^31
$\mathcal{O}(n^2)$	10^6	10^12	10^18	10^24	10^30	10^36	10^42
$\mathcal{O}(n^3)$	10^9	10^18	10^27	10^36	10^45	10^54	10^63

假如你的电脑，每秒能够执行10^9次基本运算

n	10^3	10^6	10^9	10^12	10^15	10^18	10^21
$\mathcal{O}(\log^* n)$	几乎...就是O(1)						
$\mathcal{O}(\log\log n)$	非常非常非常...接近于O(1)						
$\mathcal{O}(\log n)$	< 0.1 us = 0.0001 ms，毫无压力						
$\mathcal{O}(\sqrt{n})$		1 us	30 us	1 ms	30 ms	1 sec	30 sec
$\mathcal{O}(n)$	1 us	1 ms	1 sec	17 min	12 day	30 yr	300 c
$\mathcal{O}(n\cdot\log n)$	10 us	20 ms	30 sec	11 hr	579 day	18 century	21,000 c
$\mathcal{O}(n\cdot\sqrt{n})$	30 us	1 sec	8 hr	30 yr	10^5 c	3*10^8 c	10^13 c
$\mathcal{O}(n^2)$	1 ms	17 min	30 yr	3*10^5 c	3*10^11 c	3*10^17 c	3*10^23 c
$\mathcal{O}(n^3)$	1 sec	30 yr	3*10^8 c	3*10^17 c	3*10^26 c	3*10^35 c	3*10^44 c

实例：全国人口普查数据的排序



绪论

复杂度分析：级数

邓俊辉
deng@tsinghua.edu.cn

从跳转，到循环

❖ 没有分支转向，代码也就没有灵魂

- 函数调用（含递归）本质上都是GoTo
- 不加约束的GoTo：太任性，会毒害代码！

❖ 结构化编程：

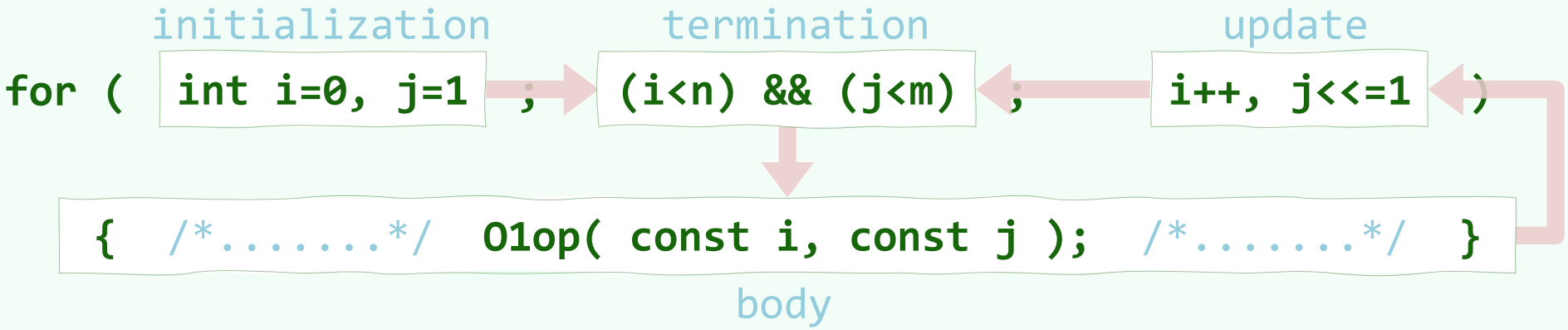
将Goto关进笼子，只隐藏，不消灭

❖ 规范的循环：

- for、while/do、repeat/until
- 本质上就是“If + GoTo”

❖ 执行过程必然呈现出某种...简明的规律

为此，需要首先在数学上做些热身...



级数

❖ **算术级数：与末项平方同阶** $\sum_{k=1}^{n} k = 1 + 2 + \ldots + n = \binom{n+1}{2} = \frac{n(n+1)}{2} = \mathcal{O}(n^2)$

❖ **幂方级数：比幂次高出一阶** $\sum_{k=0}^{n} k^d \approx \int_0^n x^d \,\mathrm{d}x = \frac{x^{d+1}}{d+1}\Big|_0^n = \frac{n^{d+1}}{d+1} = \mathcal{O}(n^{d+1})$

$$\sum_{k=1}^{n} k^2 = 1^2 + 2^2 + 3^2 + \ldots + n^2 = n(n+1)(2n+1)/6 = \mathcal{O}(n^3)$$

$$\sum_{k=1}^{n} k^3 = 1^3 + 2^3 + 3^3 + \ldots + n^3 = n^2(n+1)^2/4 = \mathcal{O}(n^4)$$

$$\sum_{k=1}^{n} k^4 = 1^4 + 2^4 + 3^4 + \ldots + n^4 = n(n+1)(2n+1)(3n^2+3n-1)/30 = \mathcal{O}(n^5)$$

❖ **几何级数：与末项同阶**

$$\sum_{k=0}^{n} a^k = a^0 + a^1 + a^2 + a^3 + \ldots + a^n = \frac{a^{n+1}-1}{a-1} = \mathcal{O}(a^n), \quad a > 1$$

$$\sum_{k=0}^{n} 2^k = 1 + 2 + 4 + 8 + \ldots + 2^n = 2^{n+1} - 1 = \mathcal{O}(2^{n+1}) = \mathcal{O}(2^n)$$

收敛级数

❖ $\sum_{k=2}^{n} 1/(k-1)/k \;=\; 1/1/2+1/2/3+1/3/4+\;\ldots\;+1/(n-1)/n \;=\; 1-1/n \;=\; \mathcal{O}(1)$

$\sum_{k=1}^{n} 1/k^2 \;=\; 1+1/2^2+1/3^2+\;\ldots\;+1/n^2 \;<\; 1+1/2^2+1/3^2+\;\ldots\;=\; \pi^2/6 \;=\; \mathcal{O}(1)$

$\sum_{k\ is\ a\ perfect\ power} 1/(k-1) \;=\; 1/3+1/7+1/8+1/15+1/24+1/26+1/31+1/35+\;\ldots\;=\; 1 \;=\; \mathcal{O}(1)$

❖ **几何分布：** $(1+2\lambda+3\lambda^2+4\lambda^3+\ldots)\cdot(1-\lambda)$

$= 1+\lambda+\lambda^2+\lambda^3+\ldots = 1/(1-\lambda) = \mathcal{O}(1), \quad 0<\lambda<1$

❖ **有必要讨论这类级数吗？难道，基本操作次数、存储单元数可能是分数?**

是的，某种意义上的确是!

发散级数

❖ **调和级数：** $\sum_{k=1}^{n} 1/k = 1 + 1/2 + 1/3 + 1/4 + \ ... \ + 1/n = \ln n + \gamma + \cancel{\mathcal{O}(1/2/n)} = \Theta(\log n)$

❖ **对数级数：** $\sum_{k=1}^{n} \ln k = \ln \prod_{k=1}^{n} k = \ln n! \approx (n+0.5) \cdot \ln n - n = \Theta(n \cdot \log n)$

❖ **对数 + 线性 + 指数：** $\sum_{k=1}^{n} k \cdot \log k \approx \int_{1}^{n} x \ln x \, \mathrm{d}x = x^2 \cdot (2 \cdot \ln x - 1)/4 \Big|_1^n = \mathcal{O}(n^2 \log n)$

$$\sum_{k=1}^{n} k \cdot 2^k = \sum_{k=1}^{n} k \cdot 2^{k+1} - \sum_{k=1}^{n} k \cdot 2^k$$

$$= (1/2) \cdot \Big(\sum_{k=1}^{n+1} (k-1) \cdot 2^{k+1} - \sum_{k=1}^{n+1} (k-1) \cdot 2^k \Big) = (n-1) \cdot 2^{n+1} + 2 = \mathcal{O}(n \cdot 2^n)$$

❖ **如有兴趣，不妨读读：Concrete Mathematics** //ex-2.35, Goldbach Theorem

【表②】级数求和表（迭代）

算数级数：与末项平方同阶	$T(n) = 1 + 2 + ... + n$	$= O(n^2)$
幂级数：比幂次高出一阶	$T(n) = 1^2 + 2^2 + 3^2 + ... + n^2$	$= O(n^3)$
	$T(n) = 1^3 + 2^3 + 3^3 + ... + n^3$	$= O(n^4)$
	$T(n) = 1^4 + 2^4 + 3^4 + ... + n^4$	$= O(n^5)$
几何级数：与末项同阶	$T(n) = 1 + 2 + 4 + ... + 2^n$	$= O(2^n)$
收敛级数：	$T(n) = 1/1/2 + 1/2/3 + 1/3/4 + ... + 1/(n-1)/n$	$= O(1)$
	$T(n) = 1 + 1/2^2 + ... + 1/n^2 < 1 + 1/2^2 + ...$	$= O(1)$
调和级数：	$h(n) = 1 + 1/2 + 1/3 + ... + 1/n$	$= \Theta(\log n)$
对数级数：	$\log 1 + \log 2 + \log 3 + ... + \log n = \log(n!)$	$= \Theta(n \log n)$

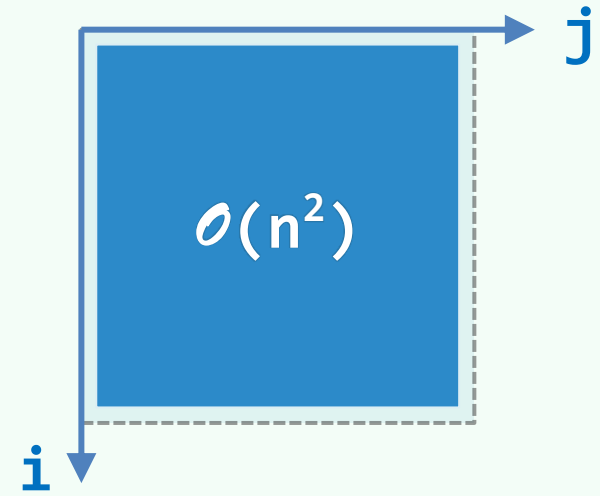
绪论

复杂度分析：可视化

邓俊辉
deng@tsinghua.edu.cn

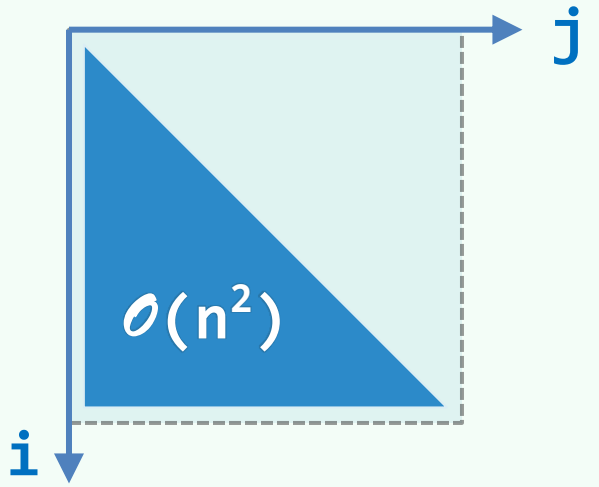
迭代 + 算术级数

```
for( int i = 0; i < n; i++ )
   for( int j = 0; j < n; j++ )
      O1op(const i, const j);
```



$$\sum_{i=0}^{n-1} n = n \times n = \mathcal{O}(n^2)$$

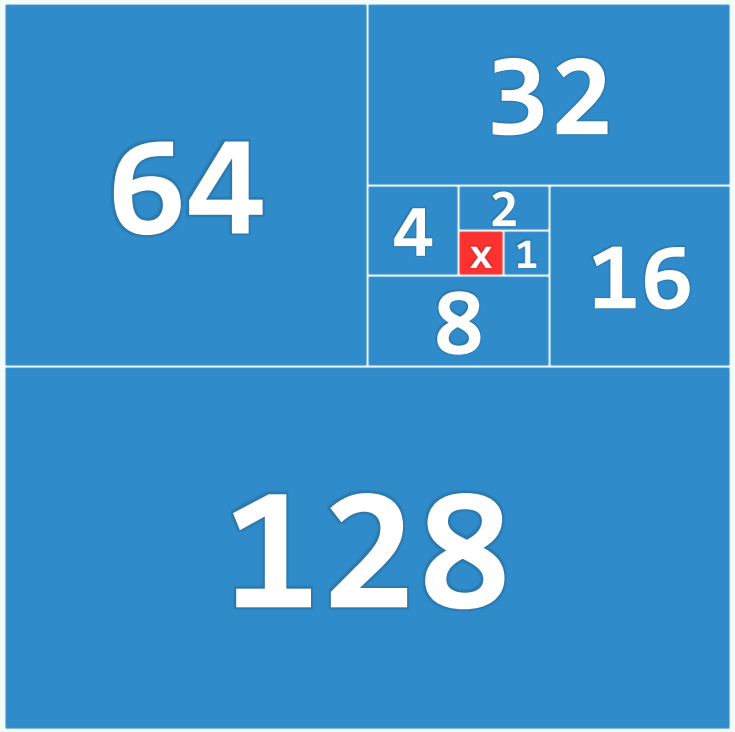
```
for( int i = 0; i < n; i++ )
   for( int j = 0; j < i; j++ )
      O1op(const i, const j);
```



$$\sum_{i=0}^{n-1} i = n \cdot (n-1)/2 = \mathcal{O}(n^2)$$

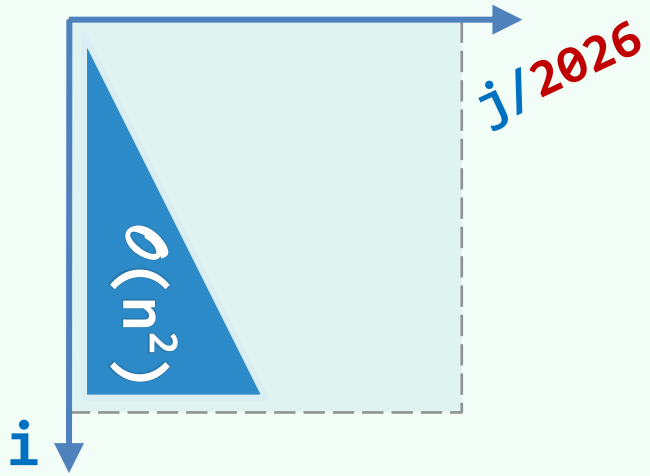
迭代 vs. 级数

```
for( int i = 1; i < n; i <<= 1 )
   for( int j = 0; j < i; j++ )
      O1op( const i, const j );
```



$$\sum_{k=0}^{\lfloor \log_2 (n-1) \rfloor} 2^k = 2^{\lceil \log_2 n \rceil} - 1 = \mathcal{O}(n)$$

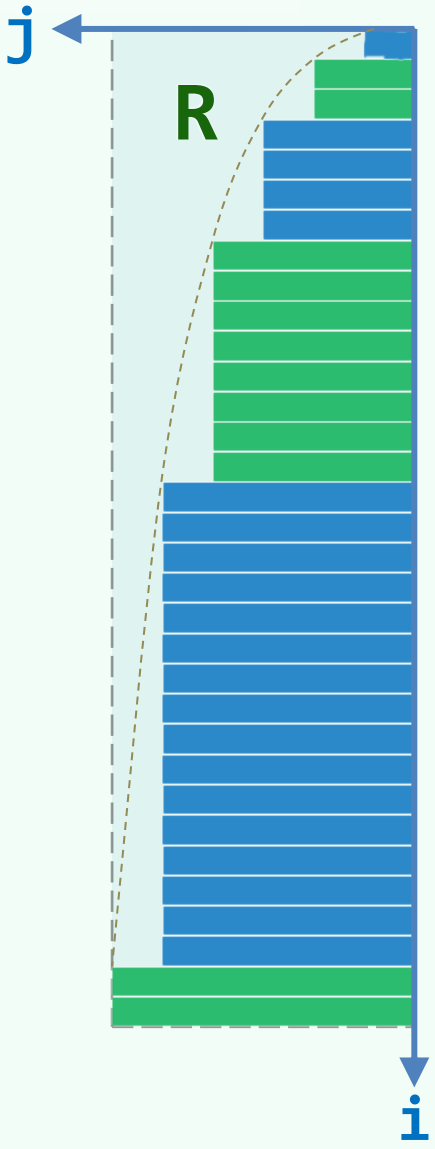
```
for( int i = 0; i < n; i++ )
   for( int j = 0; j < i; j += 2026 )
      O1op( const i, const j );
```



迭代 + 复杂级数

```
for( int i = 0; i <= n; i++ )
   for( int j = 1; j < i; j += j )
      O1op( const i, const j );
```

$$T(n) = \sum_{i=0}^{n} \lceil \log_2 i \rceil = \mathcal{O}(n \log n)$$



$$T(n) \approx \int_{x=0}^{n} \ln(x)\,\mathrm{d}x = n \cdot \ln(n) - \int_{x=0}^{n} x \cdot \mathrm{d}(\ln(x)) = n \cdot \ln(n) - n$$

$$T(n) = \sum_{k=1}^{\log n} k \cdot 2^{k-1} = \sum_{k=1}^{\log n} \sum_{i=1}^{k} 2^{k-1} = \sum_{i=1}^{\log n} \sum_{k=i}^{\log n} 2^{k-1}$$

$$T(n) \le \sum_{i=1}^{\log n} \sum_{k=1}^{\log n} 2^{k-1} \le \sum_{i=1}^{\log n} 2^{\log n} = \sum_{i=1}^{\log n} n = n \cdot \log n$$

$$T(n) \ge \sum_{i=1}^{\log n} \sum_{k=\log n}^{\log n} 2^{k-1} \ge \sum_{i=1}^{\log n} 2^{\log n - 1} = \sum_{i=1}^{\log n} n/2 = n/2 \cdot \log n$$

1．绪论

01-A.计算

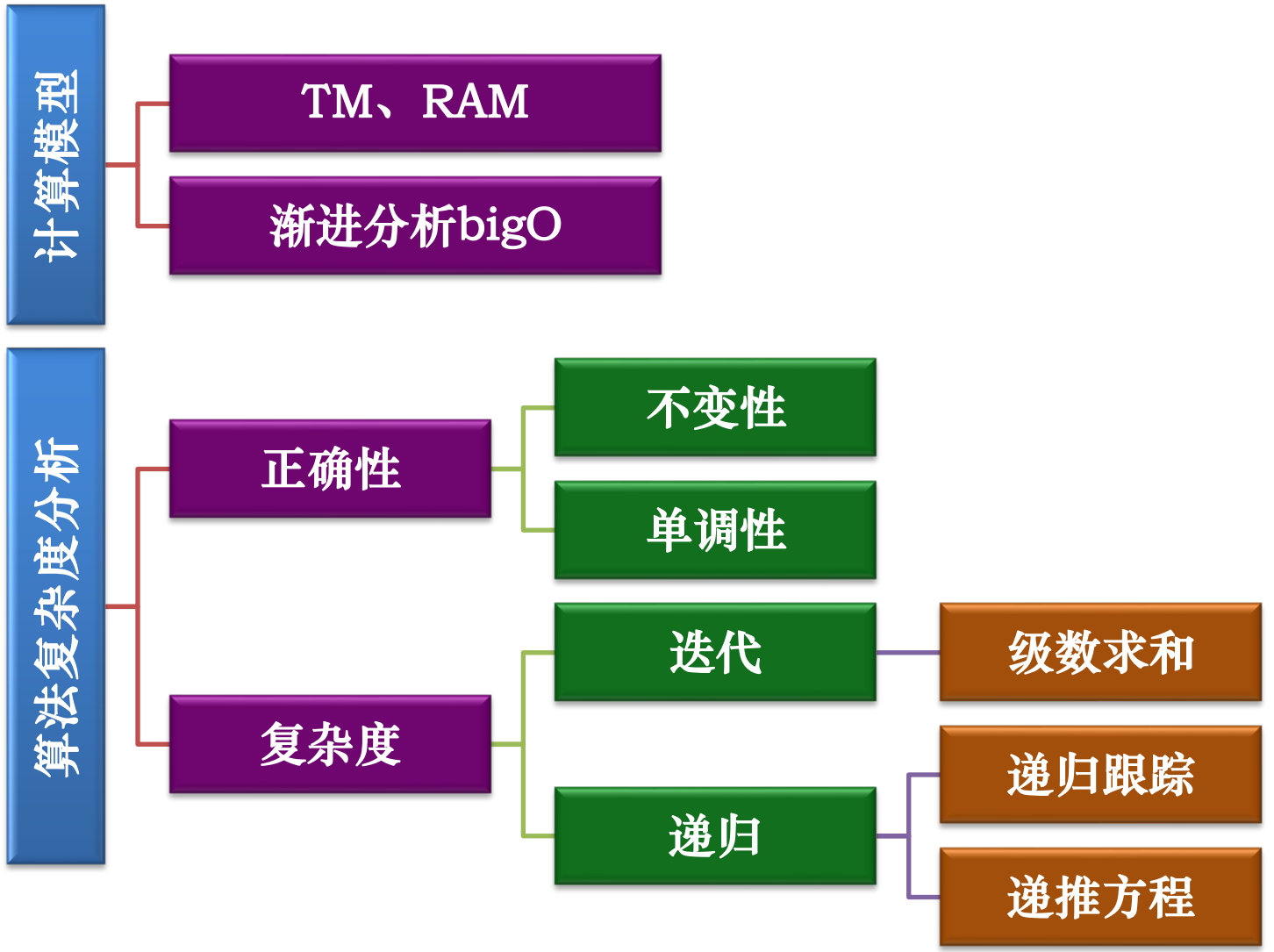
01-B.计算模型

01-C.渐进复杂度

01-D.复杂度分析

01-E.迭代与递归

01-F. 动态规划



01.Introduction

E-迭代与递归

绪论

迭代与递归：减而治之

邓俊辉
deng@tsinghua.edu.cn

整数数组之总和

❖ 还是曹冲的那堆石头

总共有多重?

❖ int SumI(int A[], int n) {

```
    int sum = 0; //累计器
    for ( int i = 0; i < n; i++ )
        sum += A[i]; //逐项累计
    return sum; //返回累计值
}
```

❖ 时间 $T(n) = \mathcal{O}(n) = \Omega(n) = \Theta(n)$

而且最好、最坏情况一样

❖ 空间呢? 你应该记得...

❖ 空间复杂度是

- 除却**输入**数据，只计入
- 在算法过程中所使用的**辅助**空间量

❖ 此处只需区区常数个辅助变量

累计不过$\mathcal{O}(1)$空间，属于**就地**算法|in-place

减而治之

❖ [decrease-and-conquer]

为求解一个大规模的问题

何不效仿愚公移山...

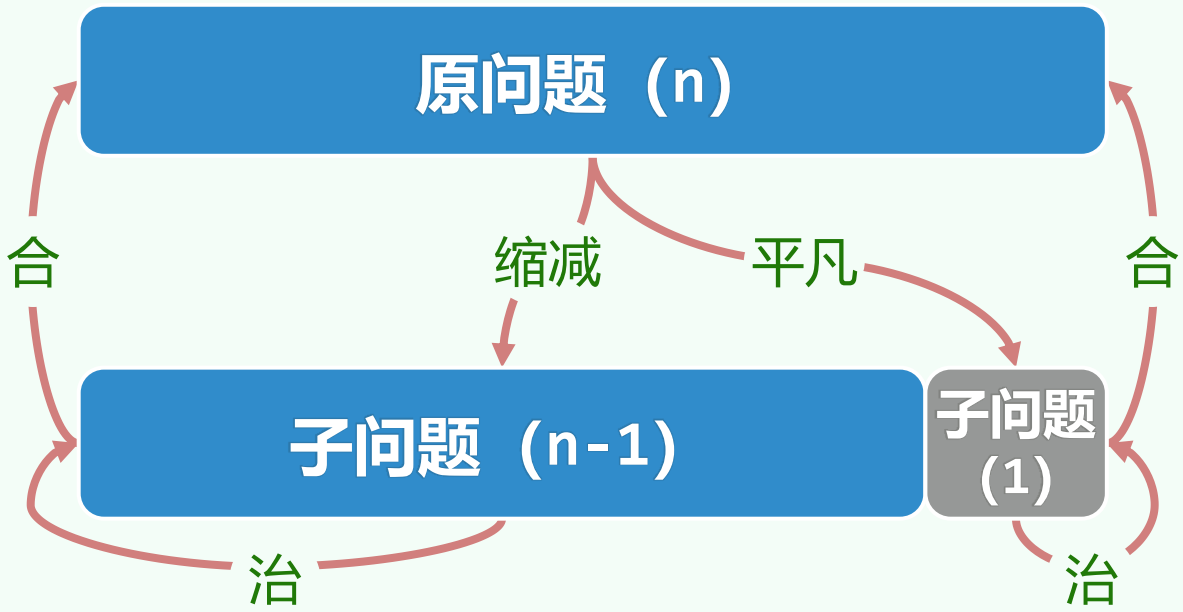
❖ 将其分解为两个子问题

- 子问题的形式不变
- 其一平凡，另一规模缩减

//单调性

❖ 递归求解非平凡的子问题

❖ 综合子问题的解，得到原问题的解



线性递归：跟踪

- `int sum( int A[], int n )`

  `{ return n < 1 ? 0 : sum(A, n-1) + A[n-1]; }`

- [trace] 绘出**调用树**|call tree

  计算过程中出现过的所有调用实例及其调用关系

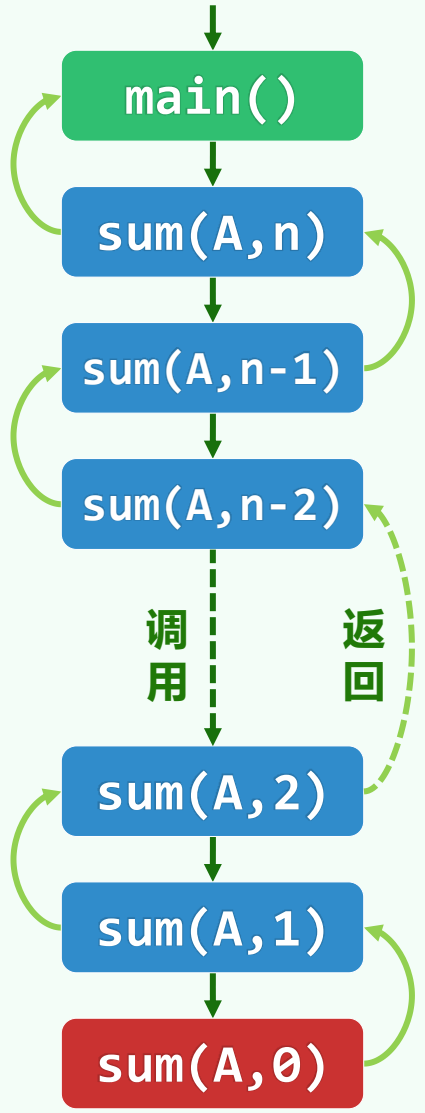
- **各实例所需时间之总和，即为整体运行时间**

  调用操作本身的成本，如何处理?

- **本例中，共计n+1个调用实例，各自只需O(1)时间**

  故总体运行时间为： $T(n) = \mathcal{O}(1) \times (n+1) = \mathcal{O}(n)$

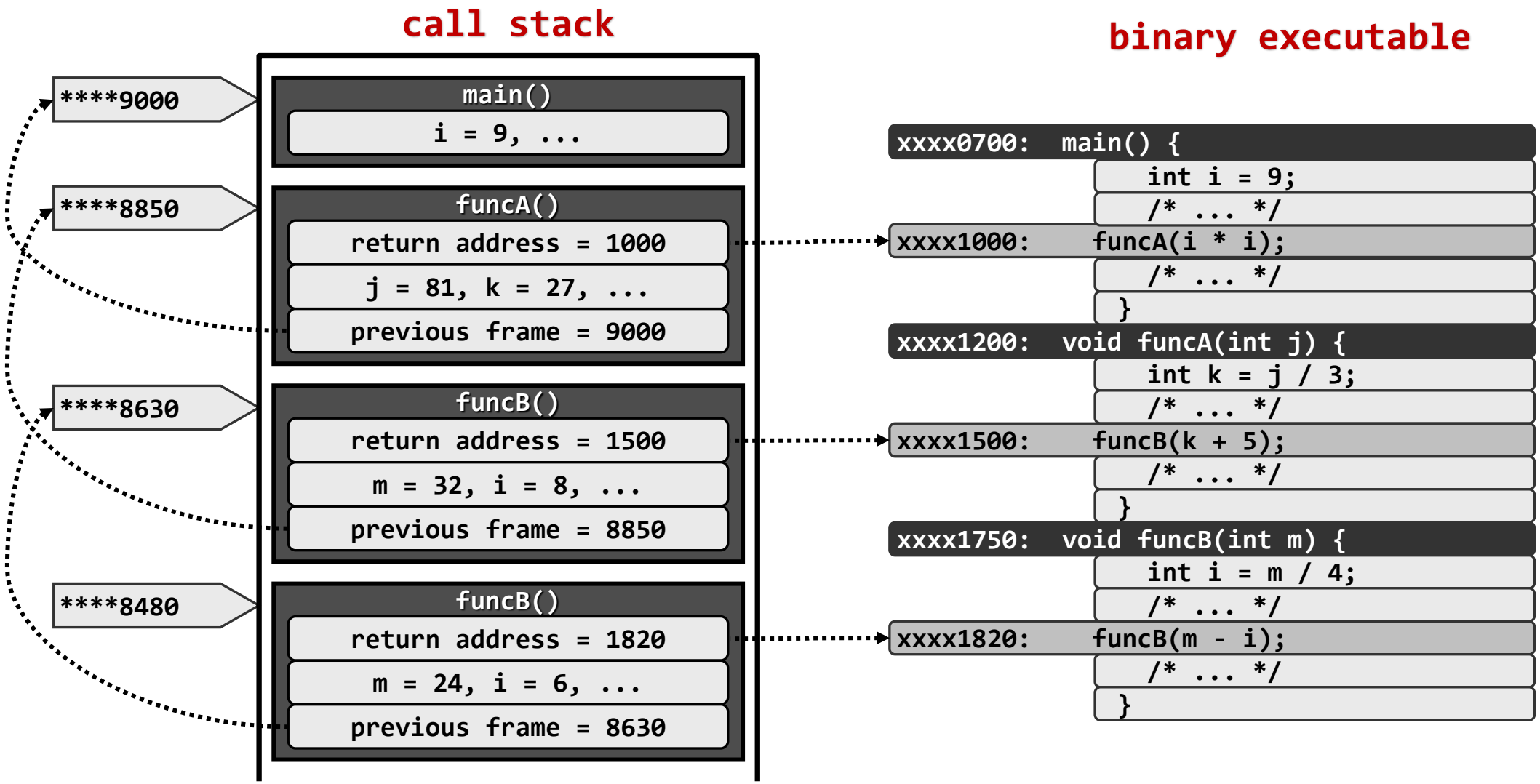
- **空间复杂度呢?**



Call Stack

- 在大多数操作系统中，每个运行的二进制程序都配有一个函数调用栈（call stack）。
  Call Stack用于根据属于同一个程序的所有函数，记录它们之间的调用关系，
  以保证被调用的函数执行完毕后才可以返回其调用函数。
- 任何一个时刻的调用栈中存放这那些被调用但尚未返回的函数实例。
  Main（）一定位于栈底，它的弹出意味着整个程序运行结束，
  并将控制权交还给操作系统。
- 在递归中，系统在后台隐式地维护call stack的过程中，
  无法区分哪些参数和变量对函数有实质性的作用，更无法优化，
  只能将描述调用关系的所有参数和变量全部入栈。
  如此一定会造成空间的浪费和时间的损耗。

递归的空间复杂度：函数调用栈



线性递归：递推

❖ 对于大规模的问题、复杂的递归算法，递归跟踪不再适用

此时可改用抽象分析的方法...

❖ [recurrence] 为求解规模为n的问题sum(A,n)，需 //T(n)

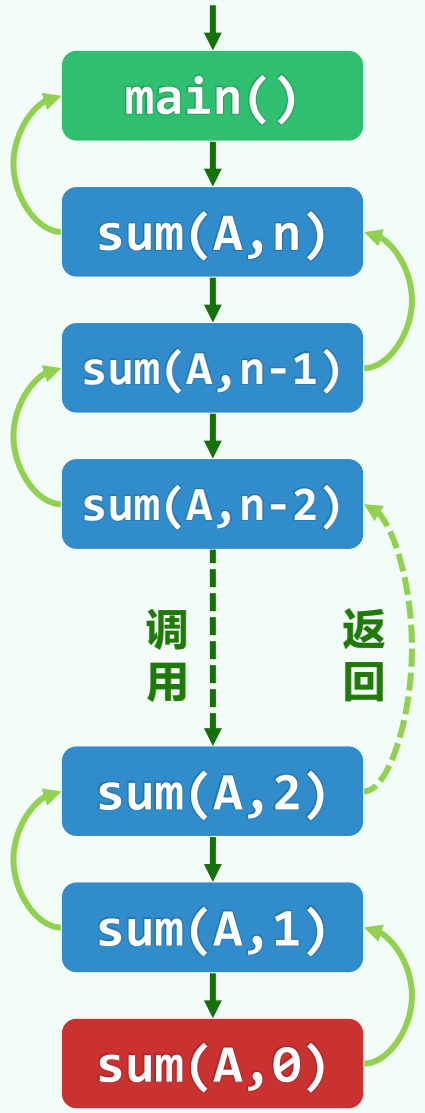
- 递归地求解规模为n-1的问题sum(A,n-1)，再 //T(n-1)
- 累加上A[n-1] //O(1)

❖ 递推方程：$T(n) = T(n-1) + \mathcal{O}(1)$ //recurrence

$T(0) \equiv \mathcal{O}(1)$ //base: sum(A, 0)

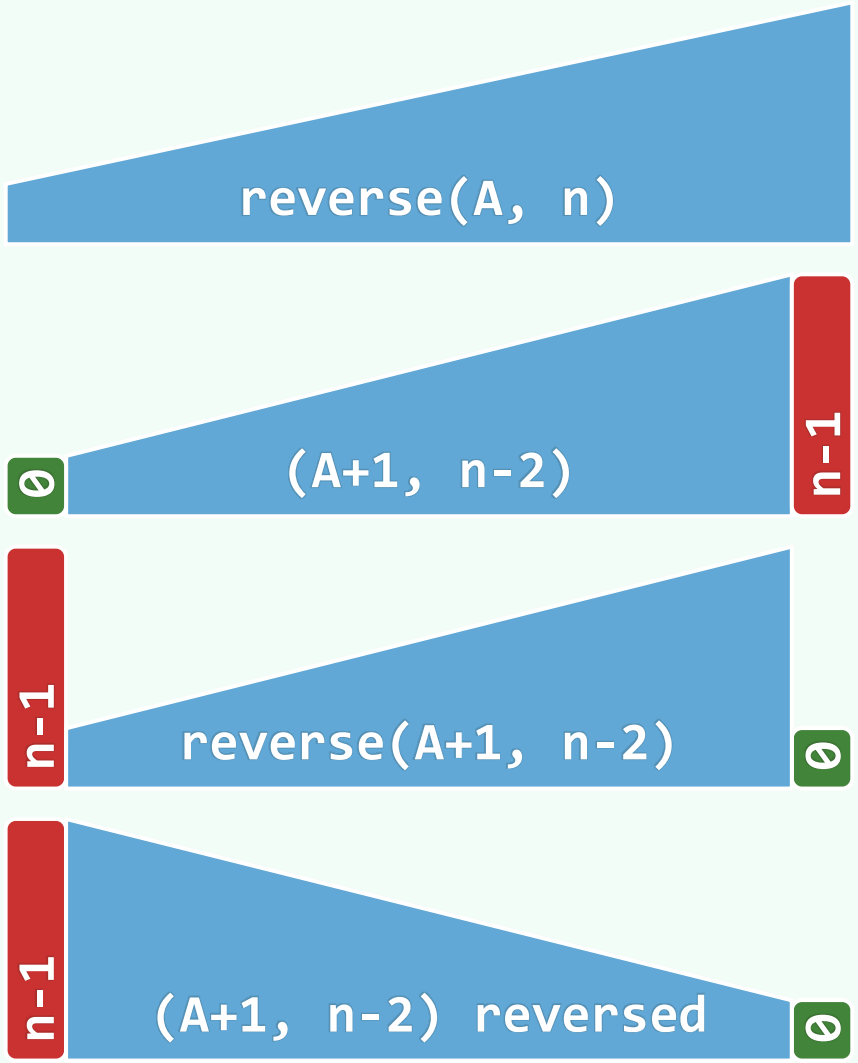
❖ 求解：$T(n) = T(n-2) + \mathcal{O}(2) = T(n-3) + \mathcal{O}(3) = \dots$

$= T(0) + \mathcal{O}(n) = \mathcal{O}(n)$



实例：数组倒置

- 减而治之
-
```
void reverse( int* A, int n ) {
    if ( n < 2 ) return;
    swap( A[0], A[n-1] );
    reverse( A+1, n-2 );
}
```
- 线性递归：$O(n)$
- 稍后：尾递归 ~ 迭代化



绪论

迭代与递归：分而治之

凡治眾如治寡，分數是也

邓俊辉
deng@tsinghua.edu.cn

分而治之

❖ [divide-and-conquer]

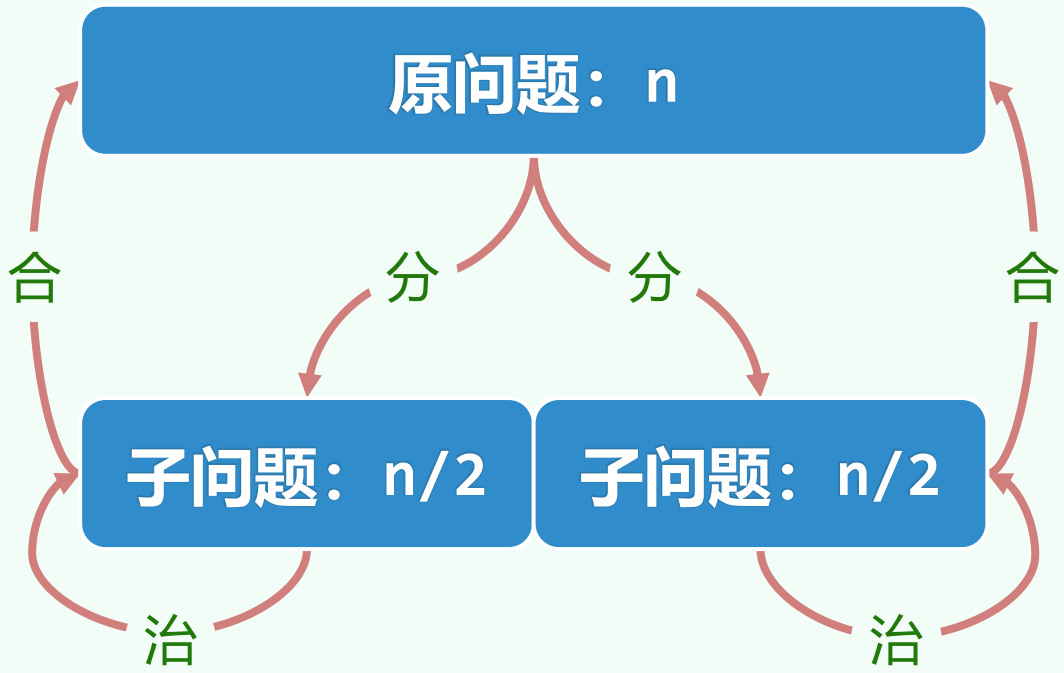
为求解一个大规模的问题，可以

将其划分为若干子问题

- 通常两个，且规模大体相当
- 子问题的形式不变

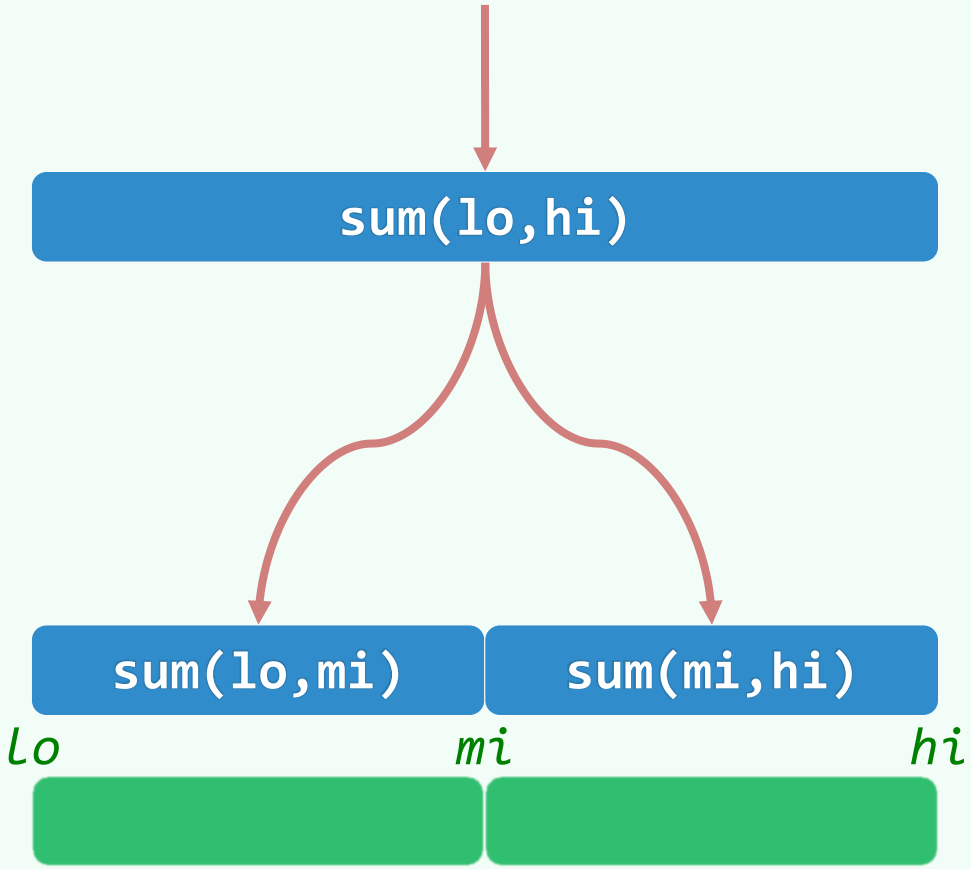
❖ 分别递归求解各个子问题

❖ 综合子问题的解，得到原问题的解



二分递归

```
int sum(int A[], int lo, int hi) {
    if ( hi-lo < 2 ) return A[lo];
    int mi = (lo+hi) / 2; //切分
    return sum(A, lo, mi) + sum(A, mi, hi);
}
```



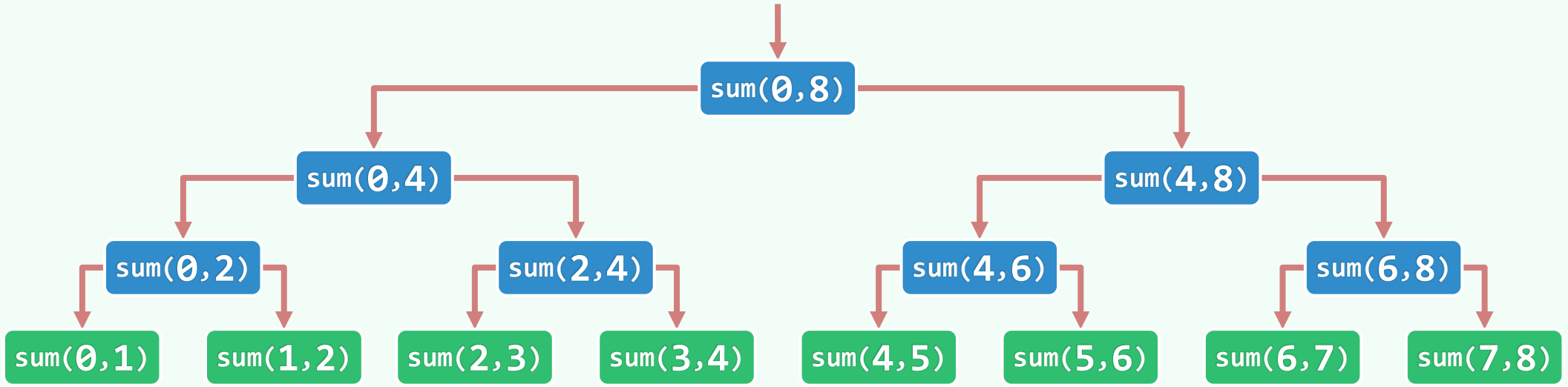
- **左闭右开的惯例：**

  指定区间范围为A[lo,hi)

- **整体求和：sum(A,0,n)**

- **原理及正确性：加法满足结合律，最终结果与执行次序无关**

二分递归：跟踪



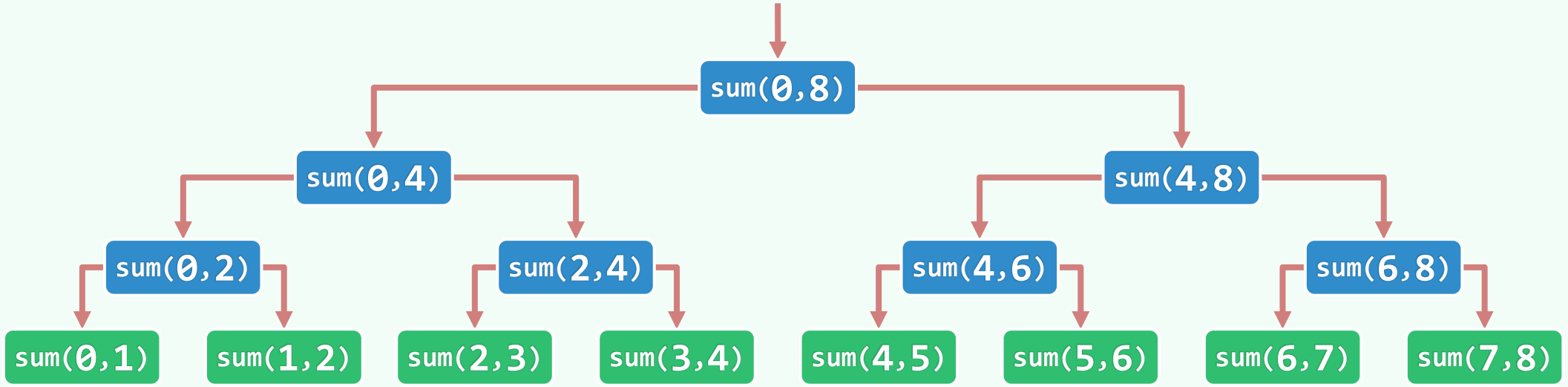
❖ 观察：每个递归实例仅需$O(1)$时间 --> 运行时间决定于**所有**递归实例的总数

$$T(n) = \mathcal{O}(1) \times (2^0 + 2^1 + 2^2 + \cdots + 2^{\log n}) = \ldots = \mathcal{O}(n)$$

观察：分层统计，递归实例数量呈**几何级数** --> 运行时间决定于**底层**递归实例的总数

❖ 每个元素，恰好**各自对应于一个递归基**...

二分递归：空间

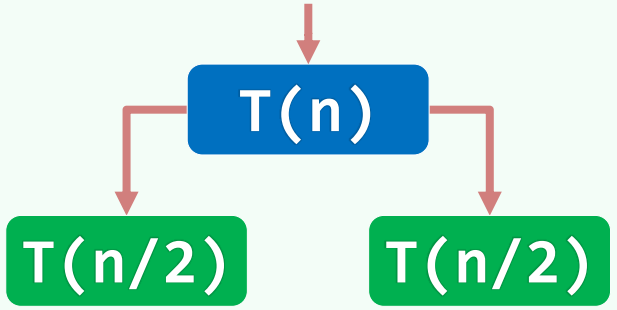


- 万变不离其宗，空间复杂度取决于空间占用量最大的时刻
- 递归算法的运行 = Theseus遍历迷宫
  空间占用量 = 绳线长度
- 在每一递归基，绳线长度都是极大
- 所有极大中的最大者，称作递归深度
  即是空间复杂度
  就此例而言，等于 $\mathcal{O}(\log n)$

Binary Recursion: Recurrence

❖ 从递推的角度看，为求解sum(A,lo,hi)，需要首先

- 递归求解sum(A,lo,mi)和sum(A,mi,hi)，进而 //2*T(n/2)
- 将子问题的解累加 //O(1)



❖ 递推方程：$T(n) = 2 \cdot T(n/2) + \mathcal{O}(1)$

$T(1) = \mathcal{O}(1)$ //base: sum(A, k, k+1)

❖ 求解：$T(n) = 4 \cdot T(n/4) + \mathcal{O}(3) = 8 \cdot T(n/8) + \mathcal{O}(7) = 16 \cdot T(n/16) + \mathcal{O}(15) = \dots$

$= n \cdot T(1) + \mathcal{O}(n-1) = \mathcal{O}(2n-1) = \mathcal{O}(n)$

【表③】递推方程表（递归）

Recurrence	Solution	Examples
T(n) = T(n-1) + O(1)	O(n)	向量求和之线性递归版
T(n) = T(n-1) + O(n)	$O(n^2)$	列表起泡排序之线性递归版
T(n) = 2*T(n-1) + 1	$O(2^n)$	Hanoi塔、Fibonacci数
T(n) = 2*T(n-1) + n	$O(2^n)$	
T(n) = T(n/2) + 1	O(logn)	向量的二分查找
T(n) = T(n/2) + n	O(n)	列表的二分查找
T(n) = 2*T(n/2) + 1	O(n)	向量求和之二分递归版
T(n) = 2*T(n/2) + n	O(nlogn)	归并排序

Master Theorem

❖ 一般地，“分而治之”策略对应的递推式是：$T(n) = a \cdot T(n/b) + \mathcal{O}(g(n))$

（原问题被分为 a 个规模为 n/b 的子任务；任务的划分、解的合并耗时 g(n)）

g(n)	$T(n)$	Example	Application
$o(n^{\log_b a})$	$\Theta(n^{\log_b a})$	$T(n) = 2 \cdot T(n/4) + \mathcal{O}(1) = \mathcal{O}(\sqrt{n})$	kd-tree
$\Theta(n^{\log_b a})$	$\Theta(n^{\log_b a} \cdot \log n)$	$T(n) = 1 \cdot T(n/2) + \mathcal{O}(1) = \mathcal{O}(\log n)$ $T(n) = 2 \cdot T(n/2) + \mathcal{O}(n) = \mathcal{O}(n \cdot \log n)$	binary search mergesort
$\omega(n^{\log_b a})$	$\Theta(f(n))$	$T(n) = 1 \cdot T(n/2) + \mathcal{O}(n) = \mathcal{O}(n)$	quickSelect (average case)

Master Theorem (1/2)

分治策略对应的递推式，通常形如：

$$T(n) = a \cdot T(n/b) + g(n)$$

//原问题被分为a个规模均为n/b的子任务

//每个递归实例自身，耗时g(n)

❖ **若** $g(n) = \Omega(n^{\log_b a + \epsilon})$ **且** $\exists c<1 \ \ s.t. \ \ a \cdot g(n/b) \leq c \cdot g(n)$ **，则** $T(n) = \Theta(g(n))$

- "binary search" in sorted lists: $T(n) = 1 \cdot T(n/2) + \mathcal{O}(n) = \mathcal{O}(n)$
- "divide-and-conquer" counting inversions: $T(n) = 2 \cdot T(n/2) + \mathcal{O}(n^2) = \mathcal{O}(n^2)$

❖ **若** $g(n) = \mathcal{O}(n^{\log_b a - \epsilon})$ **，则** $T(n) = \Theta(n^{\log_b a})$

compare >>

- recursive sum: $T(n) = 2 \cdot T(n/2) + \mathcal{O}(1) = \mathcal{O}(n)$
- kd-search: $T(n) = 2 \cdot T(n/4) + \mathcal{O}(1) = \mathcal{O}(\sqrt{n})$
- large integer multiplication: $T(n) = 3 \cdot T(n/2) + \mathcal{O}(n) = \mathcal{O}(n^{\log_2 3})$ ...

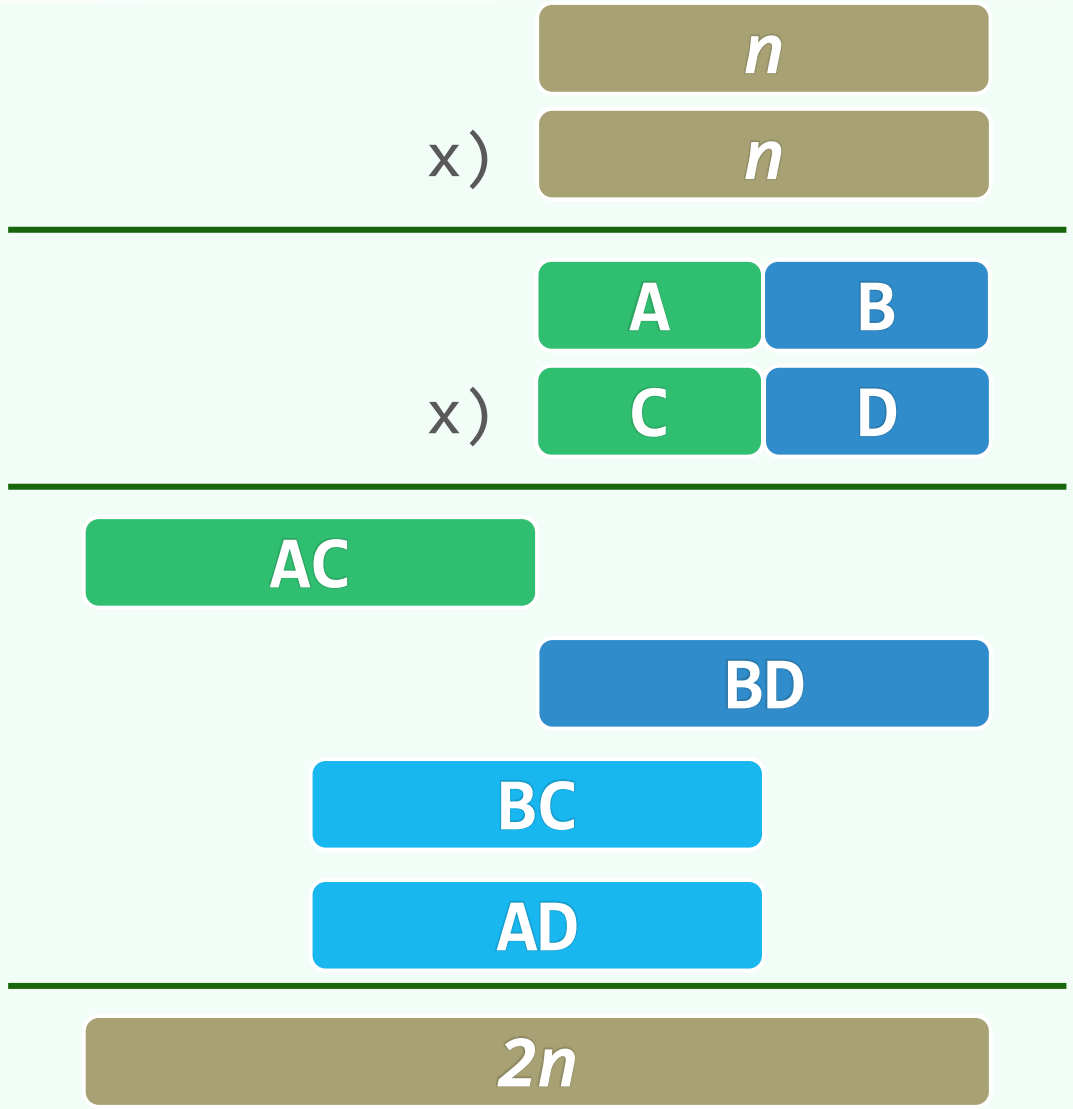
Large Integer Multiplication: Naive + DAC

$$T(n) = 4 \cdot T(n/2) + \mathcal{O}(n)$$

$$= \mathcal{O}(n^{\log_2 4})$$

$$= \mathcal{O}(n^2)$$

$$B \times C + A \times D = \ldots\ldots$$



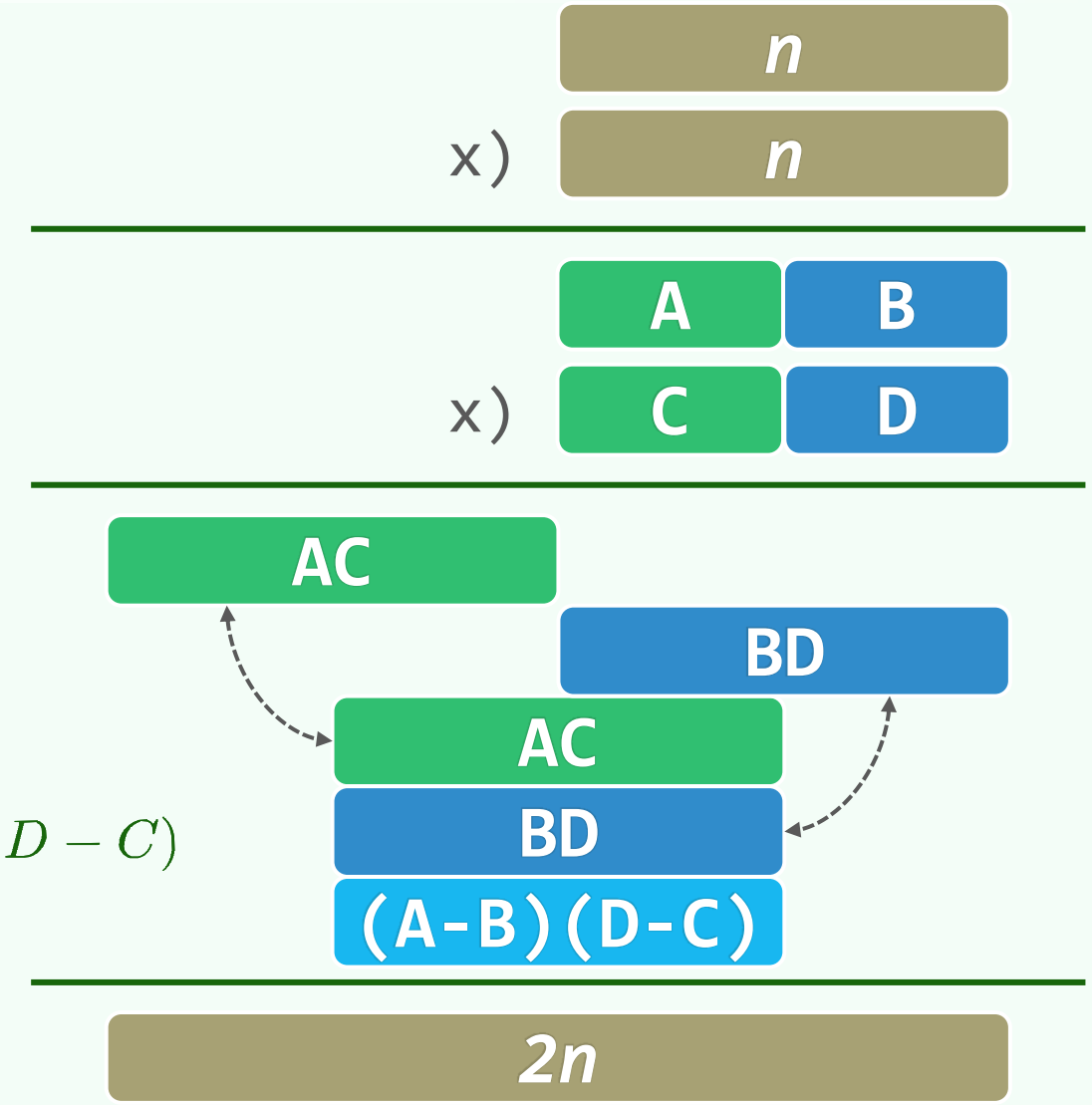
Large Integer Multiplication: Optimal

$$T(n) = 3 \cdot T(n/2) + \mathcal{O}(n)$$

$$= \mathcal{O}(n^{\log_2 3})$$

$$\approx \mathcal{O}(n^{1.585})$$

$$B \times C + A \times D \equiv A \cdot C + B \cdot D + (A - B) \times (D - C)$$



Master Theorem (2/2)

分治策略对应的递推式，通常形如：

$$T(n) = a \cdot T(n/b) + g(n)$$

//原问题被分为a个规模均为n/b的子任务

//每个递归实例自身，耗时g(n)

❖ **若** $g(n) = \Theta(n^{\log_b a} \cdot \log^k n)$ **，** $0 \leq k$

则 $T(n) = \Theta(g(n) \cdot \log n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$

- **binary search:** $T(n) = 1 \cdot T(n/2) + \mathcal{O}(1) = \mathcal{O}(\log n)$
- **mergesort:** $T(n) = 2 \cdot T(n/2) + \mathcal{O}(n) = \mathcal{O}(n \cdot \log n)$
- **C++ std::stable_sort:** $T(n) = 2 \cdot T(n/2) + \mathcal{O}(n \cdot \log n) = \mathcal{O}(n \cdot \log^2 n)$ **<< compare**

❖ **大师定理虽然强大，却没有覆盖所有的递推形式，比如**

要是...子任务的...规模不均等呢?

Akra-Bazzi Theorem

分治策略对应的递推式，更一般地形如： //k组：各含a_i个规模为$b_i n+h_i(n)$的子任务

$$T(n) = \sum_{i=1}^{k} a_i \cdot T(b_i \cdot n + h_i(n)) + g(n)$$ //任务的划分、解的合并，总共耗时g(n)

- $a_i > 0$，$0 < b_i < 1$，$|h_i(n)| = \mathcal{O}(n/\log^2 n)$
- $g(n) \geq 0$，**且有常数** $d > 0$ **使得** $|g'(n)| = \mathcal{O}(n^d)$ //多项式增长

❖ **解出满足** $\sum_{i=1}^{k} a_i \cdot b_i^p = 1$ **的** p，**则** $T(n) = \Theta(n^p \cdot (1 + \int_1^n g(u)/u^{p+1}\,\mathrm{d}u))$ //与$h_i(n)$无关

✖ **坏选择：** $T(n) = 1 \cdot T(3/4 \cdot n) + 1 \cdot T(1/4 \cdot n) + \mathcal{O}(n) = \mathcal{O}(n \log n)$ //p=1 |linearSelect

✓ **好选择：** $T(n) = 1 \cdot T(3/4 \cdot n) + 1 \cdot T(1/5 \cdot n) + \mathcal{O}(n) = \mathcal{O}(n)$ //p<1 |算法（第14章）

❖ $T(n) = 2025^2 \cdot \sum_{k=1}^{9} T(k/2025 \cdot n) + \sqrt[2025]{\prod_{k=3}^{12} n^{k^3}} = \mathcal{O}(n^3 \cdot \log n)$

绪论

迭代与递归：总和最大区间

邓俊辉
deng@tsinghua.edu.cn

问题与分析：总和最大区段

❖ 从整数序列中，找出总和最大的区段（有多个时，短者优先），例（总和最大区间有三个）：

$$\mathcal{A}[0,19) = \{\ 1,\ -2,\ \underline{7,\ 2,\ 6,\ -9,\ 5,\ 6,}\ -12,\ -8,\ \underline{13,\ 0,\ -3,\ 1,\ -2,\ 8,}\ 0,\ -5,\ 3\ \}$$

算法的魔力	总和最大区段问题求解		
	蛮力算法BF	O(n^3)	Brute Force
	递增策略IC	O(n^2)	Incremental Strategy
	分治策略DC	O(n*logn)	Divide-And-Conquer
	减治策略LS	O(n)	Decrease-And-Conquer (Linear Scan)

问题 + 蛮力算法

❖ 从整数序列中，找出总和最大的区间（有多个时，短者、靠后者优先）

$$\mathcal{A}[0,19) = \{ 1, -2, 7, 2, 6, -9, 5, 6, -12, -8, 13, 0, -3, 1, -2, 8, 0, -5, 3 \}$$

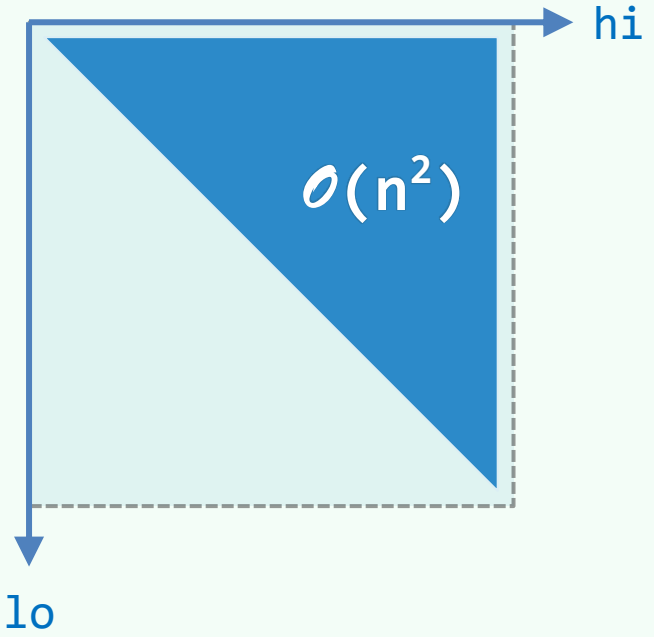
```
❖ int gs_BF( int A[], int n ) { //蛮力策略：O(n^3)
    int gs = 0; //目前已知的最大和：零宽度区间，总和为零
    for ( int lo = 0; lo < n; lo++ ) //枚举所有的
       for ( int hi = lo; hi < n; ) { //O(n^2)个区间!
          int s = sum( A, lo, ++hi ); //用O(n)时间求和
          gs = max( gs, s ); //择优更新
       }
    return gs;
} //gs_BF
```

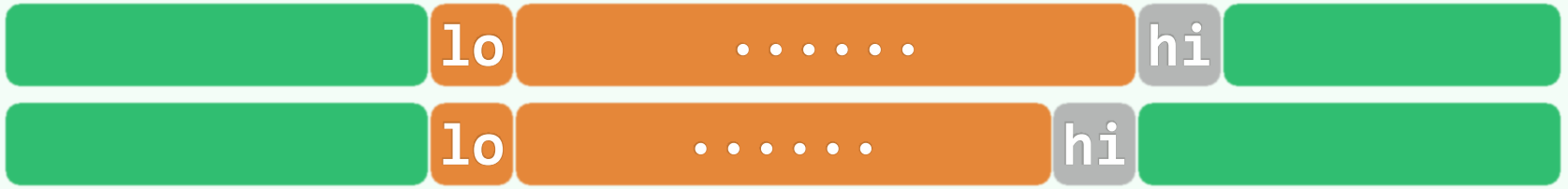


递增策略

❖ 考查所有起点**相同**的区间：它们的总和之间具有**相关性**...

❖
```
int gs_IC( int A[], int n ) { //递增策略：O(n^2)
   int gs = 0; //目前已知的最大和：零宽度区间，总和为零
   for ( int lo = 0; lo < n; lo++ ) //枚举所有起始于lo
      for ( int s = 0, hi = lo; hi < n; ) { //终止于hi的区间
         s += A[ hi++ ]; //递增地得到其总和：O(1)
         gs = max( gs, s ); //择优更新
      }
   return gs;
} //gs_IC
```





分而治之：前缀 + 后缀：$\mathcal{A}[lo, hi) = \mathcal{A}[lo, mi) \cup \mathcal{A}[mi, hi) = \mathcal{P} \cup \mathcal{S}$

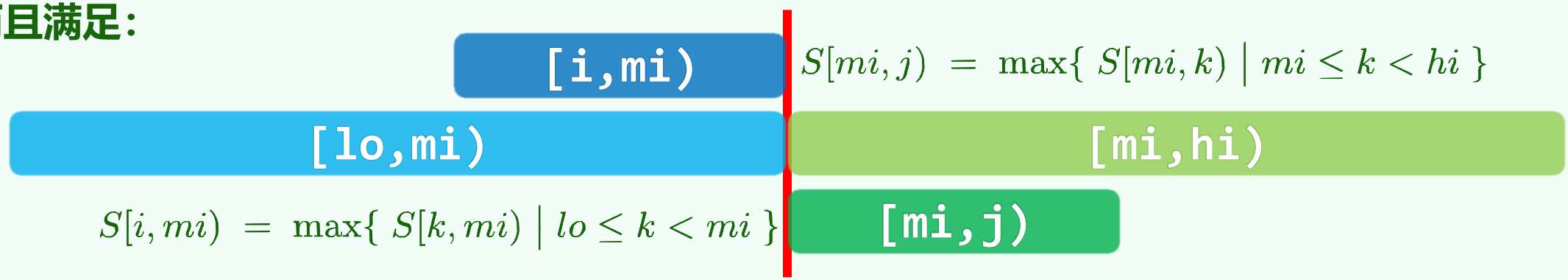
❖ 借助递归，便可求得$\mathcal{P}$、$\mathcal{S}$内部的GS；而剩余的实质任务无非是...

考查那些跨越切分线的区间...

❖ 沿着切分线，这类区间必被分割为非空的前缀、后缀：

$$\mathcal{A}[i, j) = \mathcal{A}[i, mi) + \mathcal{A}[mi, j), \quad i < mi < j$$

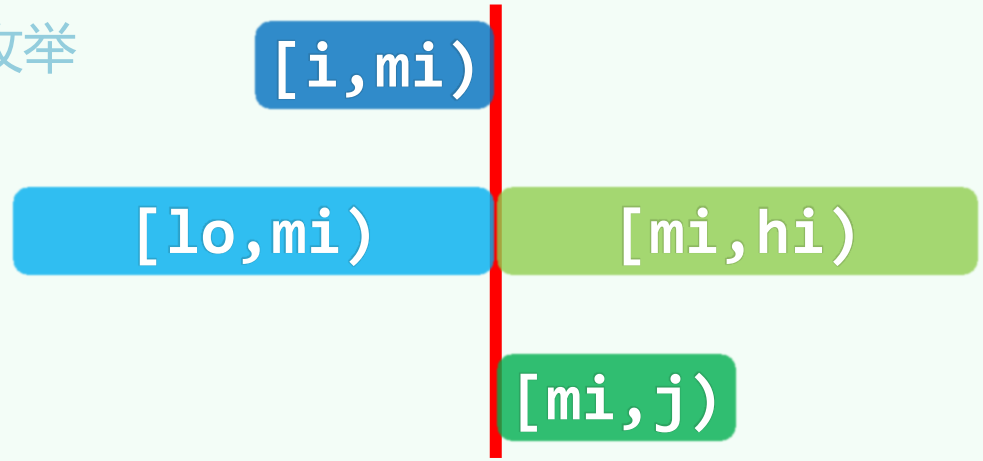
而且满足：



❖ 可见，二者均可独立计算，且累计耗时不过 $\mathcal{O}(n)$；于是总体复杂度也优化为 $\mathcal{O}(n \cdot \log n)$

分而治之：实现

```
int gs_DC( int A[], int lo, int hi ) { //Divide-And-Conquer: O(n*logn)
   if ( hi - lo < 2 ) return A[lo]; //递归基
   int mi = (lo + hi) / 2; int gsL = 0, gsR = 0; //前缀|后缀中的最大后缀|前缀
   for ( int s = 0, i = mi-1; lo <= i; i-- ) //枚举
      gsL = max( gsL, s += A[i] ); //所有[i,mi)
   for ( int s = 0, j = mi; j < hi; j++ ) //枚举
      gsR = max( gsR, s += A[j] ); //所有[mi,j)
   return max( gsL + gsR, max( gs_DC(A, lo, mi), gs_DC(A, mi, hi) ) ); //递归
} //gs_DC
```



减而治之：原理

- 关注GS的终点，比如第一个就是n
- 由短渐长，考查所有的后缀[k,n)

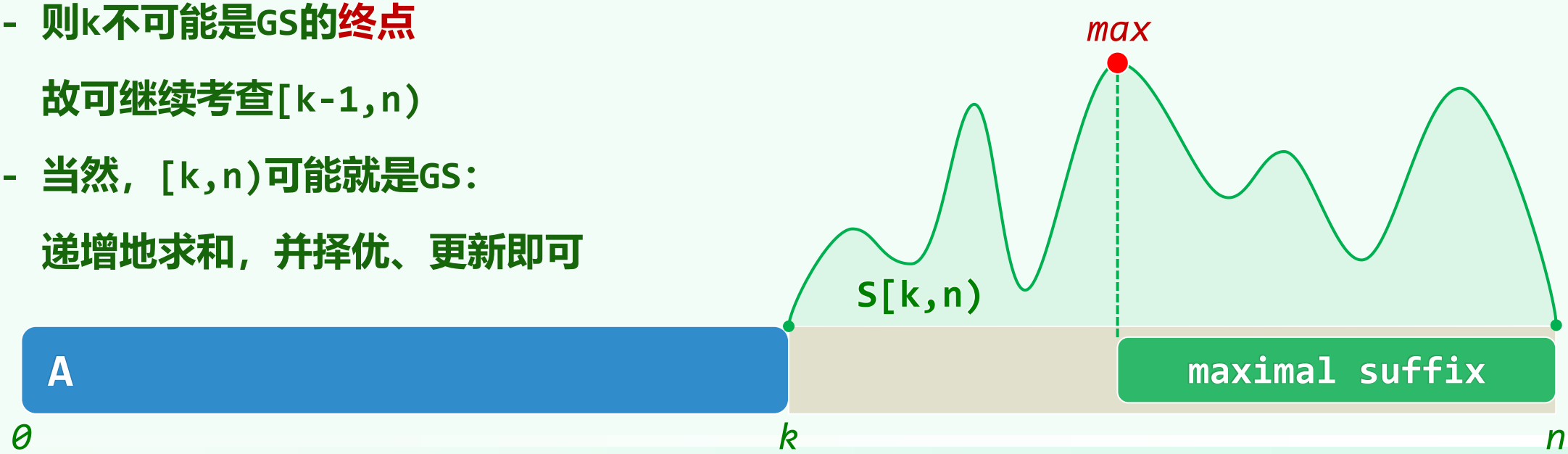
  视S[k,n)的正负，无非两种情况
- 若S[k,n) > 0
  - 则k不可能是GS的终点

    故可继续考查[k-1,n)
  - 当然，[k,n)可能就是GS：

    递增地求和，并择优、更新即可
- 反之，若S[k,n) <= 0
  - 则（以n为终点的）更长的后缀不可能是GS

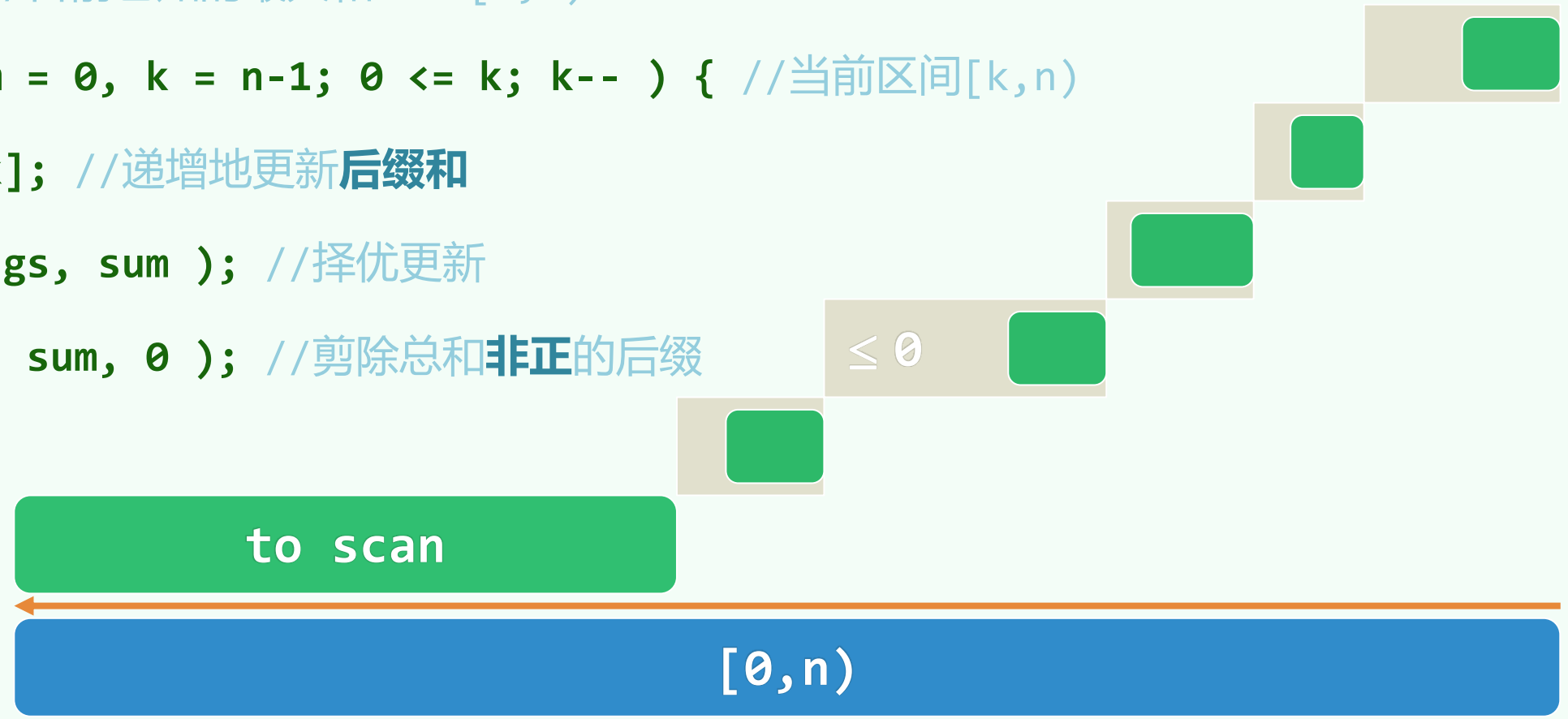
    故可剪除[k,n) —— 令n = k
  - 当然，k仍可能是GS的终点：

    重复以上，继续考查（以k为终点的）后缀



减而治之：实现

```
int gs_LS( int A[], int n ) { //减治策略：O(n)
   int gs = 0; //目前已知的最大和 = S[n,n) = 0
   for ( int sum = 0, k = n-1; 0 <= k; k-- ) { //当前区间[k,n)
      sum += A[k]; //递增地更新后缀和
      gs = max( gs, sum ); //择优更新
      sum = max( sum, 0 ); //剪除总和非正的后缀
   }
   return gs;
} //gs_LS
```



I	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
A[I]	1	-2	7	2	6	-9	5	6	-12	-8	13	0	-3	1	-2	8	0	-5	3

$$\mathcal{A}[0,19) = \{\ 1,\ -2,\ 7,\ 2,\ 6,\ -9,\ 5,\ 6,\ -12,\ -8,\ 13,\ 0,\ -3,\ 1,\ -2,\ 8,\ 0,\ -5,\ 3\ \}$$

gs	i	A[i]	S
1	19		0
3	18	3	3
	17	-5	-2 → 0
	16	0	0 → 0
8	15	8	8
	14	-2	6
	13	1	7
	12	-3	4
	11	0	4
17	10	13	17

gs	i	A[i]	S
17	10	13	17
	9	-8	9
	8	-12	-3 → 0
	7	6	6
	6	5	11
	5	-9	2
	4	6	8
	3	2	10
	2	7	17
	1	-2	14
	0	1	15

```
int gs_LS( int A[], int n ) { //L
    int gs = A[0], s = 0, i = n;
    while ( 0 < i-- ) { //在当前区间
        s += A[i]; //递增地累计总和
        if ( gs < s ) gs = s; //并打
        if ( s <= 0 ) s = 0; //剪除
    }
    return gs;
}
```

1．绪论

01-A.计算

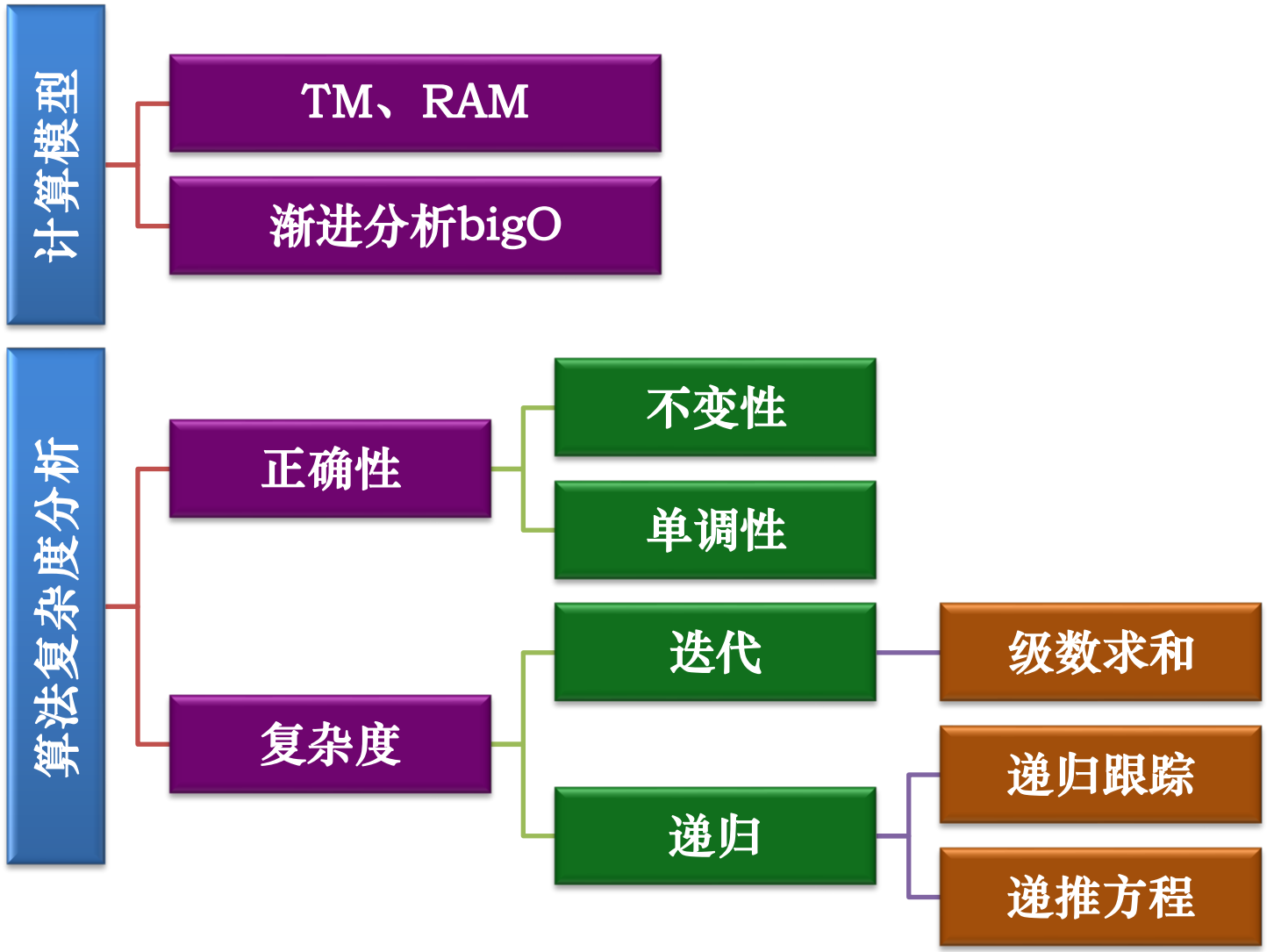
01-B.计算模型

01-C.渐进复杂度

01-D.复杂度分析

01-E.迭代与递归

01-F. 动态规划



01.Introduction

F-动态规划

绪论

动态规划：Fibonacci数

邓俊辉
deng@tsinghua.edu.cn

二分递归

$$fib(n) = fib(n-1) + fib(n-2)$$

0 1 1 2 3 5 8 13 21 34 55 89 ...

```
uint64_t fib(n) { return (2 > n) ? n : fib(n-1) + fib(n-2); } //为何这么慢?
```

❖ **复杂度:** $T(n) = T(n-1) + T(n-2) + 1$

- **若令** $S(n) = [T(n)+1]/2$ **，则** $S(0) = 1 = fib(1)$ **，** $S(1) = 1 = fib(2)$
- **故有** $S(n) = S(n-1) + S(n-2) = fib(n+1)$

$$T(n) = 2 \cdot S(n) - 1 = 2 \cdot fib(n+1) - 1 = \mathcal{O}(\phi^n)$$

- **其中** $\phi = (1+\sqrt{5})/2 \approx 1.618$

❖ **也可以...猜...** $T(n) = \mathcal{O}(\alpha^n - 1)$

于是 $\mathcal{O}(\alpha^n) = \mathcal{O}(\alpha^{n-1}) + \mathcal{O}(\alpha^{n-2})$ **，** $\alpha^2 = \alpha^1 + \alpha^0$ **，解得** $\alpha = (1+\sqrt{5})/2 = \phi$

封底估算

$$\phi^{36} \approx 2^{25}$$

$$\phi^{43} = 2^{43*25/36} = 2^{1075/36} \approx 2^{1080/36} = 2^{30} \approx 10^{9}\ flo \sim 1\ sec$$

$$\phi^{5} \approx 10^{1}$$

$$\phi^{67} \approx 10^{14}\ flo \sim 10^{5}\ sec \approx 1\ day$$

$$\phi^{92} \approx 10^{19}\ flo \sim 10^{10}\ sec \approx 10^{5}\ day \approx 3\ century$$

记忆化：原理

- 低效的根源在于

  有大量重复的递归实例

  - 先后出现的实例，共计$\mathcal{O}(\phi^n)$个
  - 而去重之后，总共不过 $\mathcal{O}(n)$ 种

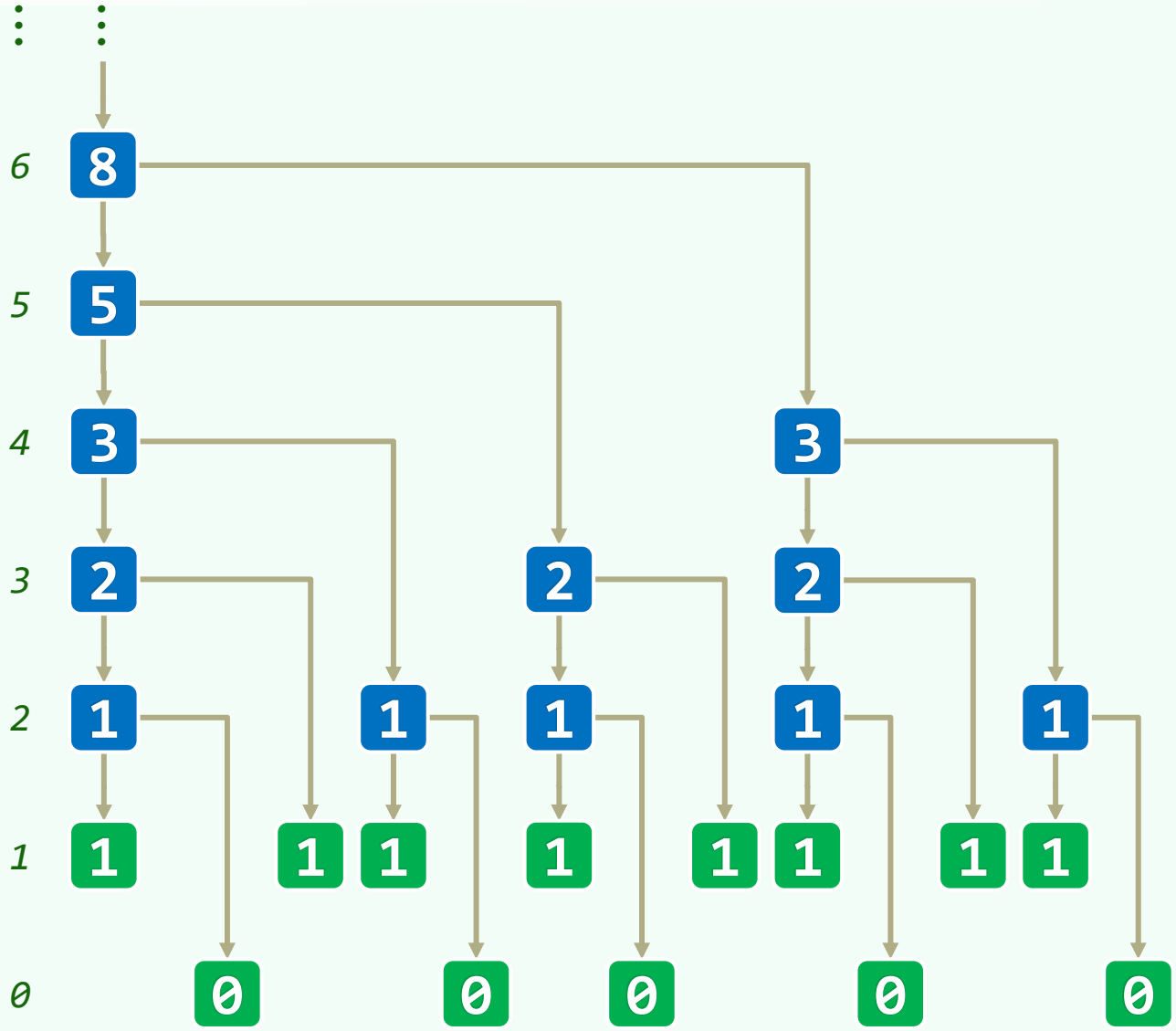
- 再三出现、结果不变的计算，何必重复?

- [memoization]

  - 谁先算过，便将答案记下
  - 其他各位，直接CX就好!

- 好记性不如烂笔头

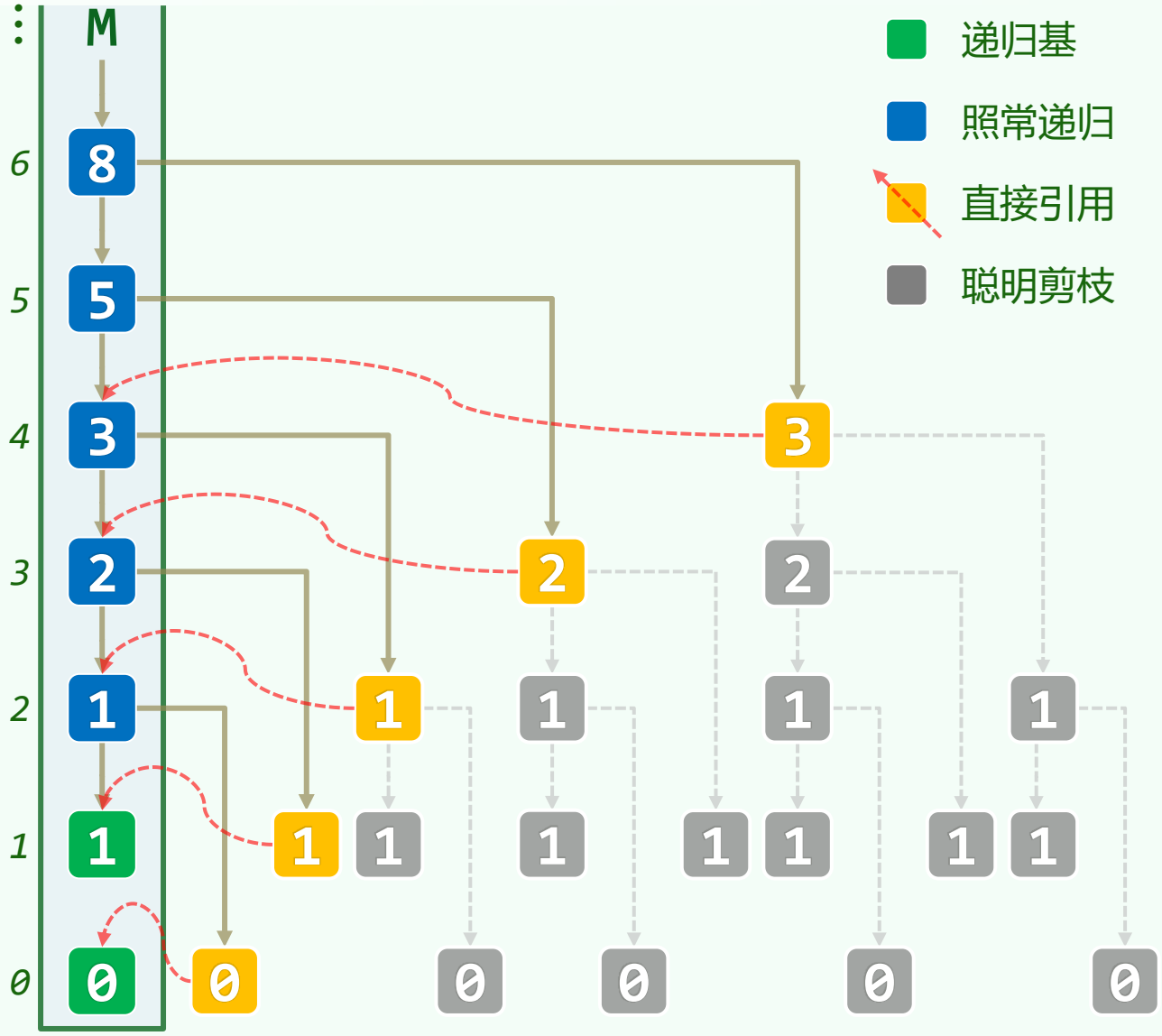
  准备一张速查表...



记忆化：实现

- 计算之前先查表，如已算过，便直接引用
  - extern uint64_t M[];
  - 前两项作为递归基，初始化为0、1
  - 其余各项均为UINT64_MAX
- 每个子问题仅算一次，算罢随即记入速查表

```
uint64_t fibM( int n ) { //O(n)
   if ( UINT64_MAX == M[n] )
      M[n] = fibM(n-1) + fibM(n-2);
   return M[n]; //直接引用
} //fibM
```



记忆化：一般模式

```
def f(n)
    if ( n < 1 ) return trivial( n );

    return f(n-X) + f(n-Y)*f(n-Z);
```

```
T M[ N ]; #init. with UNDEFINED
M[0] = trivial(0); //recursion base

def f(n)

 # recurs only at 1st call
    if ( M[n] == UNDEFINED )
        M[n] = f(n-X) + f(n-Y)*f(n-Z);

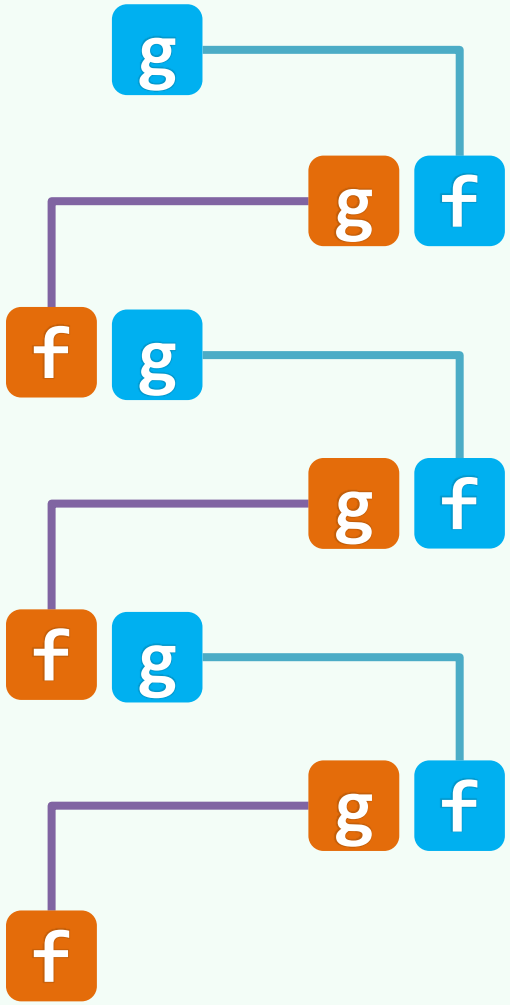
    return M[n];
```

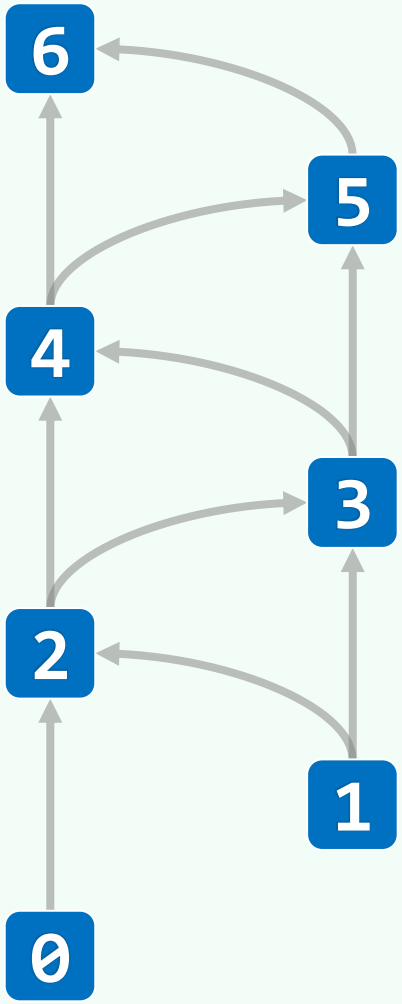
Dynamic Programming：颠倒计算方向

- 由自顶而下递归，改为自底而上迭代

-
```
f = 1; g = 0; //fib(-1), fib(0)
while ( 0 < n-- ) {
   g = g + f;
   f = g - f;
}
return g;
```

- T(n) = O(n)，而且仅需O(1)空间！





绪论

动态规划：最长公共子序列

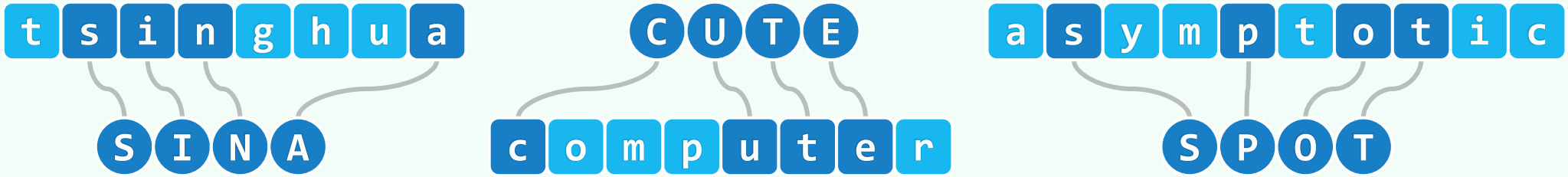
Make it work, make it right, make it fast.

世上一切都无独有偶，为什么你与我却否？

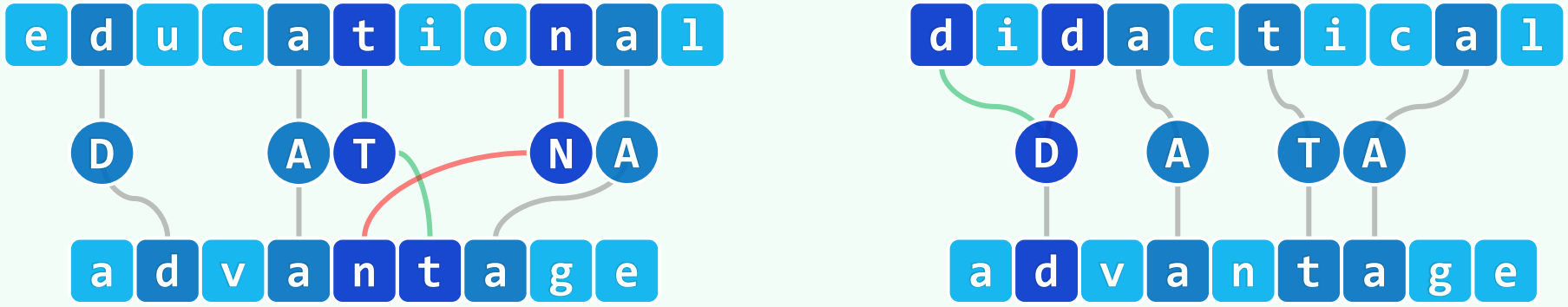
邓俊辉
deng@tsinghua.edu.cn

最长公共子序列

❖ subsequence：由序列中若干字符，按原相对次序构成



❖ longest common subsequence：两个序列之间公共子序列中的最长者



多解　　歧义

递推

❖ 对于序列A[0,n)和B[0,m)

LCS(n,m)无非三种情况

❖ 若n=0或m=0，则取作

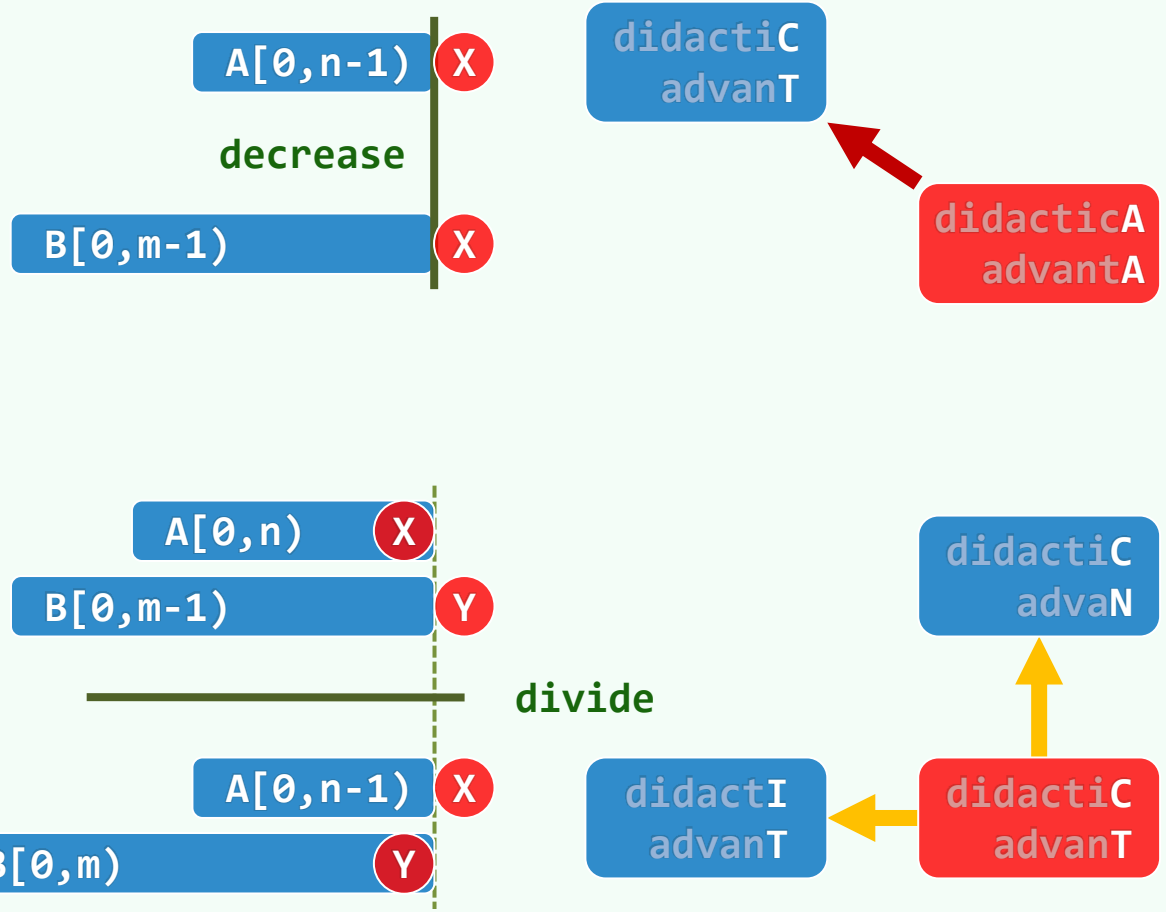
[递归基] 空序列（长度为零）

❖ 若A[n-1]='X'=B[m-1]，则取作

[减治] LCS(n-1,m-1) + 'X'

❖ 若A[n-1]='X' ≠ 'Y'=B[m-1]，则取作

[分治] LCS(n,m-1)与LCS(n-1,m)中的更长者



描述：伪代码

```
Input: two strings A and B of length n and m resp.,
Output: (the length of) the longest common subsequence of A and B

lcs(A[], n, B[], m)

    Compare the last characters of A and B, i.e., A[n-1] and B[m-1]

    If A[n-1] = B[m-1]
        Compute x = lcs(A, n-1, B, m-1) recursively and return 1 + x

    Else
        Compute x = lcs(A, n-1, B, m) & y = lcs(A, n, B, m-1) and return max(x, y)

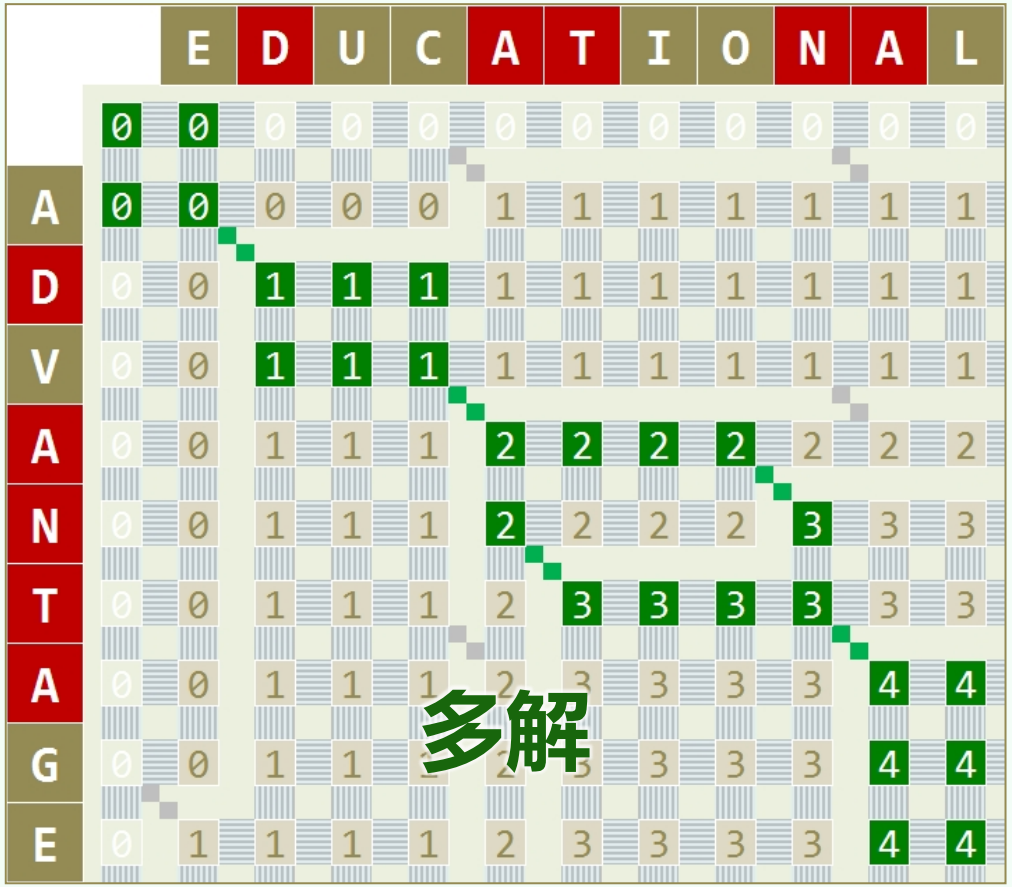
    As the recursion base, return 0 when either n or m is 0
```

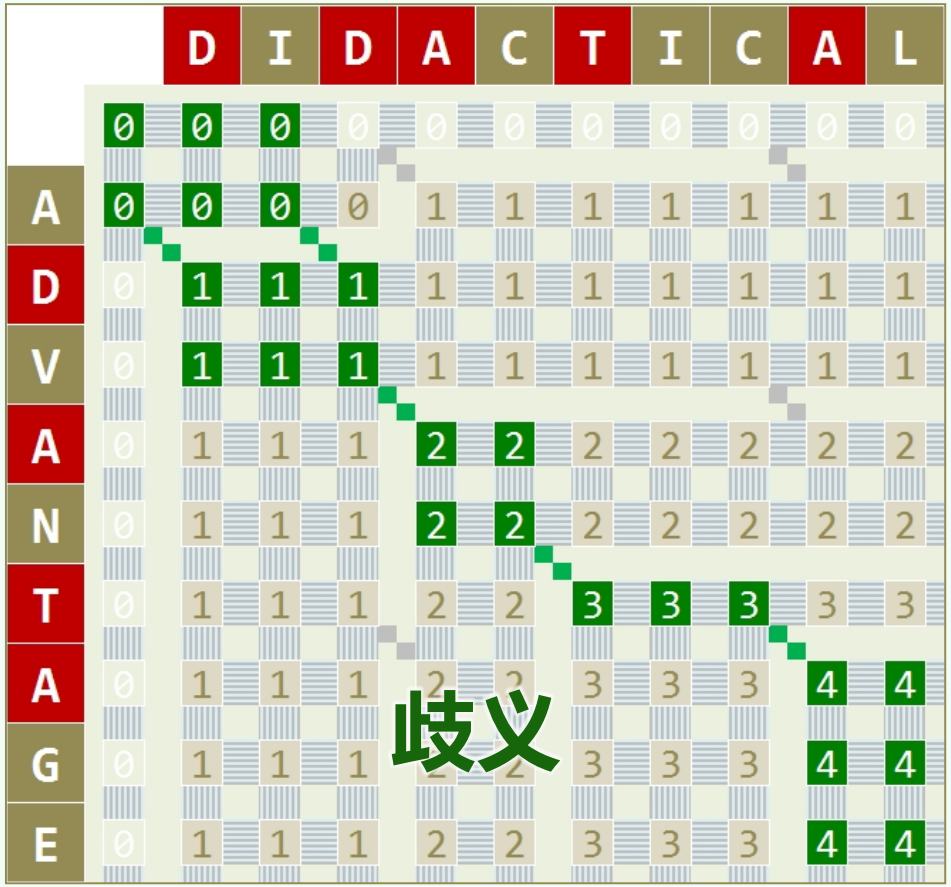
实现：递归版

```
uint32_t lcs( const char* A, int n, const char* B, int m ) {
   if (n < 1 || m < 1) //trivial cases
      return 0;
   if ( A[n-1] == B[m-1] ) //decrease & conquer
      return 1 + lcs( A, n-1, B, m-1 );
   else //divide & conquer
      return max( lcs( A, n-1, B, m ), lcs( A, n, B, m-1 ) );
} //lcs
```

子问题空间

❖ 子问题按递推关系串接的通路，便对应于LCS的解；沿途减治递推的次数，即是LCS长度





指数复杂度

❖ 总共无非 $n \times m$ 种子问题

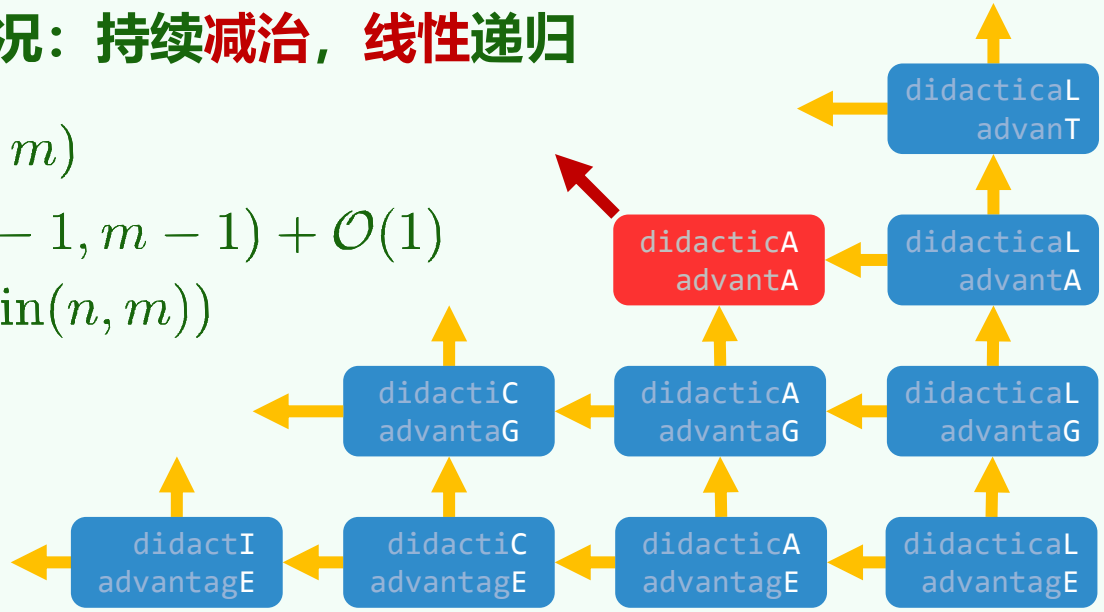
可见有大量重复的递归

❖ 任何子问题 $\langle a, b \rangle$，都可能

重复计算 $\binom{n+m-a-b}{n-a} = \binom{n+m-a-b}{m-b}$ 次

❖ 最好情况：持续减治，线性递归

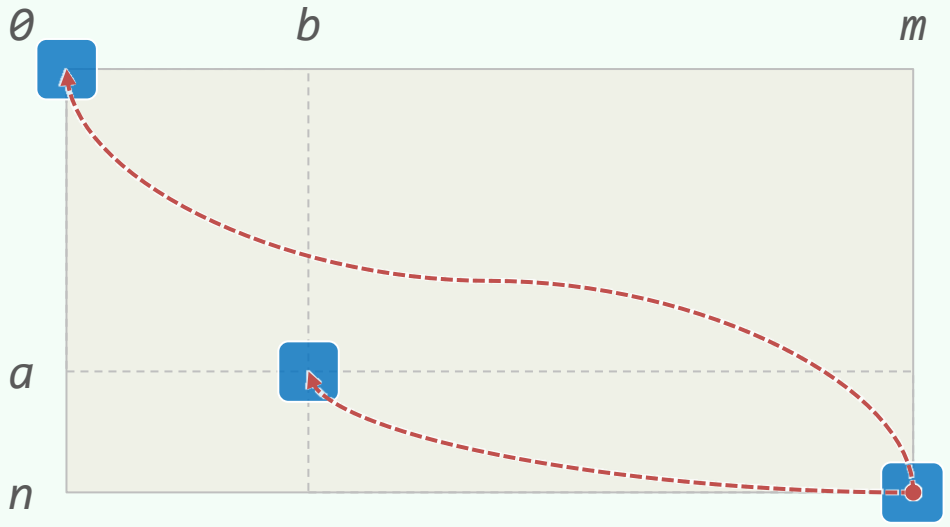
$$\begin{aligned} & T(n,m) \\ = \; & T(n-1,m-1)+\mathcal{O}(1) \\ = \; & \mathcal{O}(\min(n,m)) \end{aligned}$$



❖ 最坏情况：持续分治，二分递归

$$\begin{aligned} T(n,m) &= T(n-1,n)+T(n,m-1)+\mathcal{O}(1) \\ &= \tbinom{n+m}{n}-\mathcal{O}(1) \end{aligned}$$

$$\begin{aligned} T(n,n) &= \tbinom{2n}{n}-\mathcal{O}(1) \\ &\approx 4^n/\sqrt{\pi \cdot n} = \Omega((4-\epsilon)^n) \end{aligned}$$



实现：记忆化版

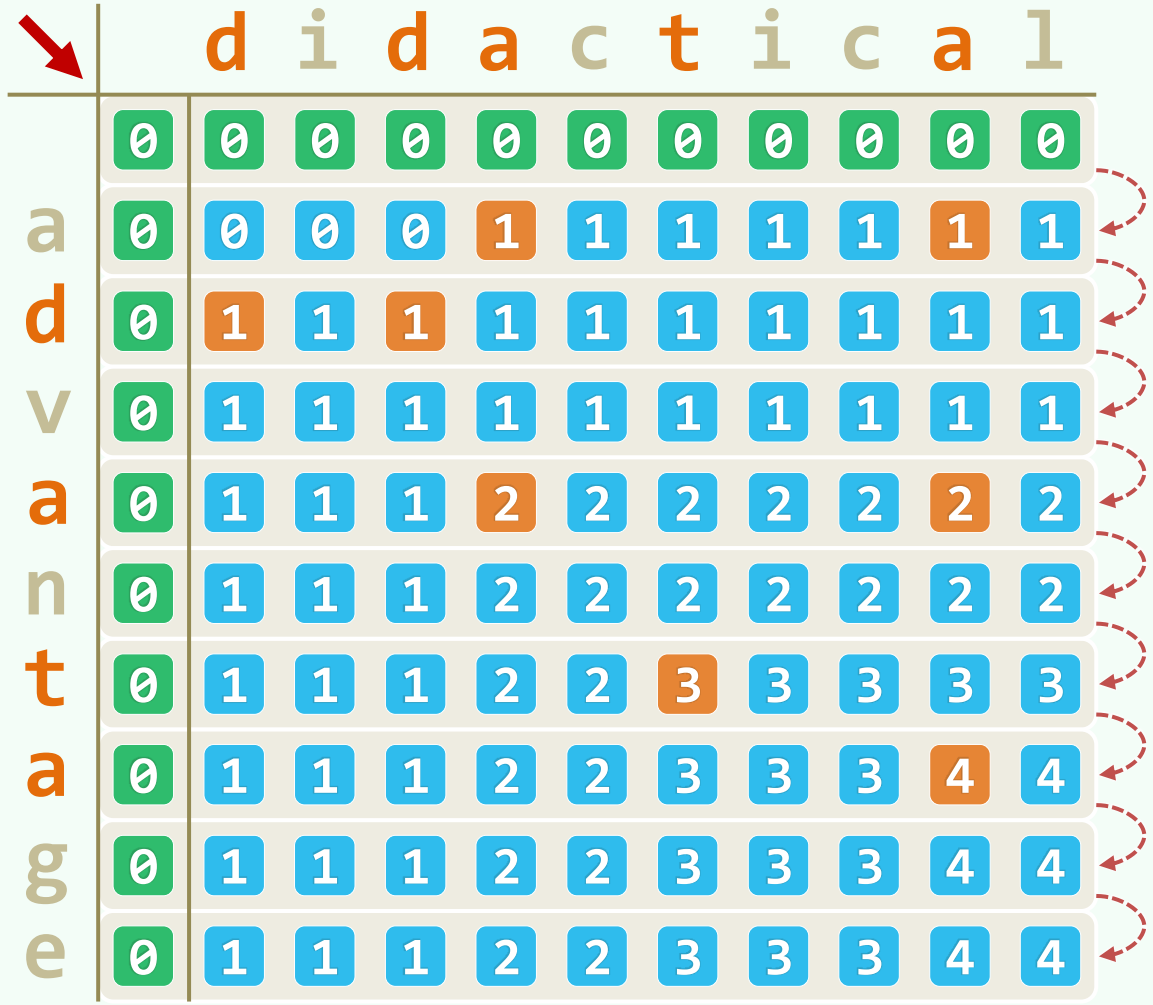
❖ 每个子问题至多递归一次，时间复杂度降至 $\mathcal{O}(n \cdot m)$

用到二维的快查表，空间复杂度也悄然增至 $\mathcal{O}(n \cdot m)$ ——如何降低?

❖
```
extern uint32_t** M; //(n+1)*(m+1)的二维快查表：0行、0列已初始化为0，其余为UINT_MAX
uint32_t lcsM(const char* A, int n, const char* B, int m) {
   if (UINT_MAX == M[n-1][m-1]) //每个子问题只会在第一次求值时，才需照常递归计算
      M[n-1][m-1] = (A[n-1] == B[m-1]) ?
                    1 + lcsM(A, n-1, B, m-1) :
                    max(lcsM(A, n-1, B, m), lcsM(A, n, B, m-1));
   return M[n-1][m-1]; //否则，可以直接引用快查表
} //lcsM：既然递归基的答案已经预存，故可省去对应的判断
```

动态规划

- 翻转思路：从递归基开始，逐步递推
  - 以第0行为基础：LCS(a,0) = 0
    依次解出第1行的各个子问题：LCS(a,1)
  - 以第1行为基础
    依次解出第2行的各个子问题：LCS(a,2)
  - ...
  - 直到LCS(n-1,m-1)
- 子问题各计算一次，总共不过 $\mathcal{O}(n \cdot m)$ 时间
- 更重要的是，始终只需滚动地记忆两行
  于是，空间复杂度随即降至 $\mathcal{O}(\max(n, m))$
- 稍作优化，还可进一步降至 $\mathcal{O}(\min(n, m))$



实现：迭代（动态规划）版

```
uint32_t lcs( const char* A, int n, const char* B, int m) {
   if (n < m) { swap(A, B); swap(n, m); } //make sure m <= n
   uint32_t* lcs1 = new uint32_t[m+1]; //the current two rows are
   uint32_t* lcs2 = new uint32_t[m+1]; //buffered alternatively
   memset( lcs1, 0x00, sizeof(uint32_t) * (m+1) ); lcs2[0] = 0;//sentinels
   for ( int i = 0; i < n; swap( lcs1, lcs2 ), i++ )
      for ( int j = 0; j < m; j++ )
         lcs2[j+1] = ( A[i] == B[j] ) ? 1 + lcs1[j] : max( lcs2[j], lcs1[j+1] );
   uint32_t solu = lcs1[m]; delete[] lcs1; delete[] lcs2; return solu;
} //lcs
```

1. 绪论

01-A.计算

01-B.计算模型

01-C.渐进复杂度

01-D.复杂度分析

01-E.迭代与递归

01-F. 动态规划

